

Operating System Concepts

A. Silberschatz, P. B. Galvin, and G. Gagne, 10th edition

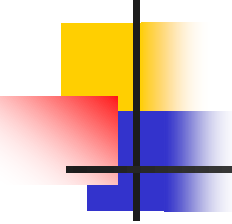
Chapter 5: CPU Scheduling

Mei-Ling Chiang, Professor

**Dept. of Information Management
National Chi-Nan University, Taiwan, ROC**

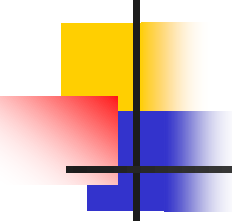


2025/10/23



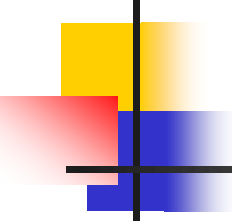
Chapter Objectives

- To introduce process scheduling, which is the basis for multiprogrammed OSes
- To describe various CPU-scheduling algorithms
- To discuss evaluation criteria for selecting a CPU-scheduling algorithm for a particular system
- To examine the scheduling algorithms of several OSes



Chapter 5: CPU Scheduling

- **5.1 Basic Concepts** 
- **5.2 Scheduling Criteria** 
- **5.3 Scheduling Algorithms** 
- **5.4 Thread Scheduling** 
- **5.5 Multiple-Processor Scheduling** 
- **5.6 Real-Time CPU Scheduling** 
- **5.7 Operating System Examples** 
- **5.8 Algorithm Evaluation** 



5.1 Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- Almost all computer resources are **scheduled before use.**
- CPU scheduling is central to OS design.

5.1.1 CPU-I/O

Burst Cycle

- CPU – I/O Burst Cycle – Process execution consists of a **cycle** of **CPU execution** and **I/O wait**. (Fig. 5.1)

•
•
•
load store
add store
read from file

} CPU burst

wait for I/O

} I/O burst

store increment
index
write to file

} CPU burst

wait for I/O

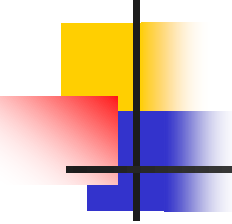
} I/O burst

load store
add store
read from file

} CPU burst

wait for I/O

} I/O burst
•
•
•

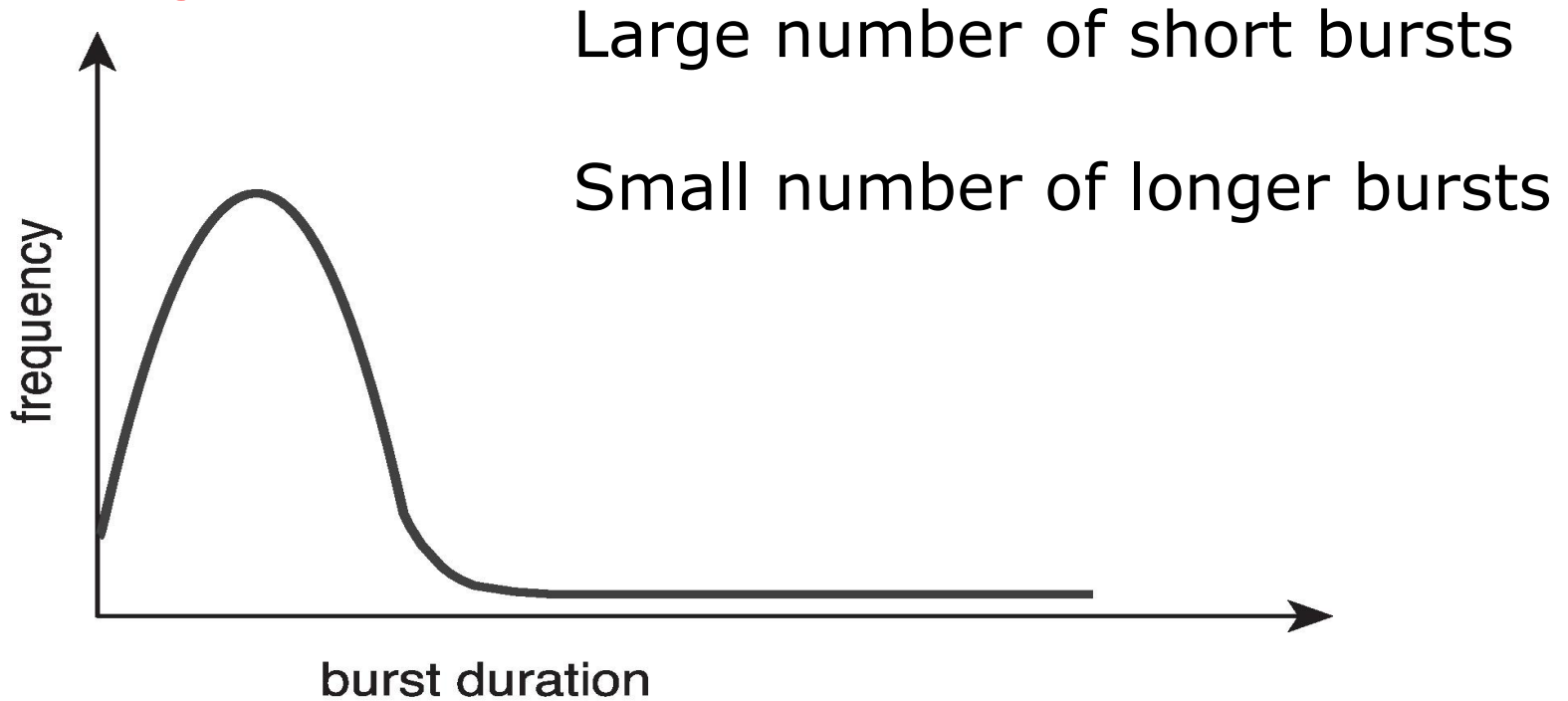


CPU-I/O Burst Cycle (Cont.)

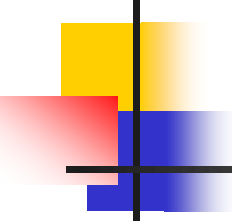
- CPU burst distribution (Fig. 5.2)
- An **I/O-bound program** typically has many **short** CPU bursts.
- A **CPU-bound program** might have a few **long** CPU bursts.

Histogram of CPU-burst Durations (Fig. 5.2)

frequency

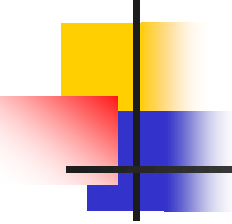


burst duration (milliseconds)



5.1.2 CPU Scheduler

- Whenever CPU becomes **idle**, OS must **select** one of the processes in the **ready queue** to be executed.
 - The selection process is carried out by the **CPU scheduler**.
 - A ready queue can be implemented by a **FIFO** queue, a **priority** queue, a **tree**, or an **unordered linked** list.

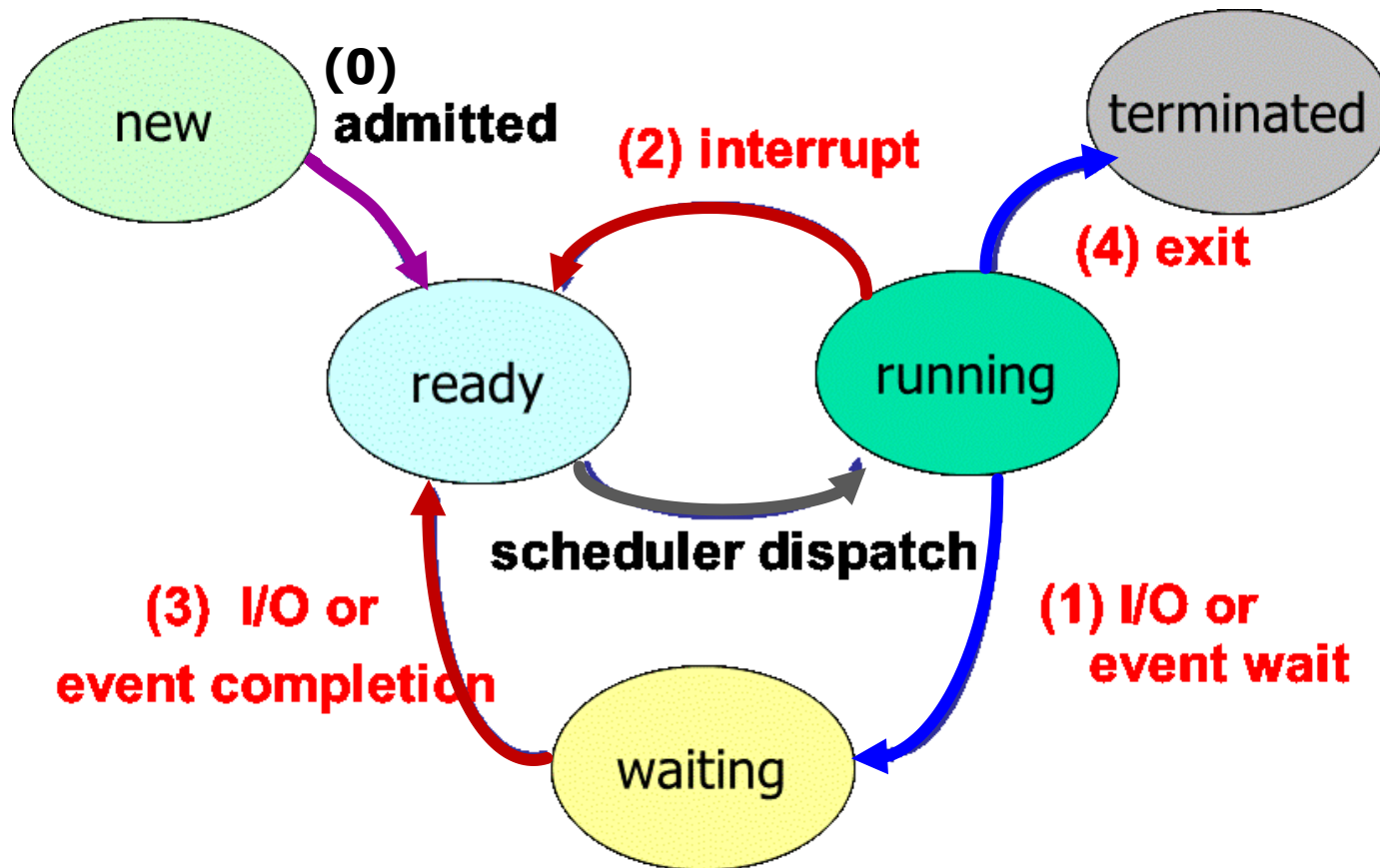


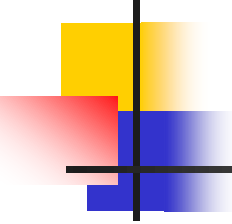
5.1.3 Preemptive & Nonpreemptive Scheduling

- CPU-scheduling decisions may take place when a process:
 - ① Switches from running to waiting state.
 2. Switches from running to ready state.
 3. Switches from waiting to ready state.
 - ④ Terminates.
- When scheduling takes place only under circumstances 1 and 4, it is **nonpreemptive** or **cooperative**; otherwise, it is **preemptive**.

Process State Diagram

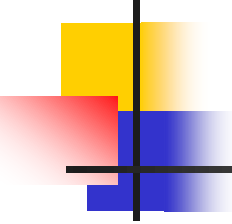
Scheduling points: preemptive (0,1,2,3,4)
nonpreemptive (0,1,2,3,4)





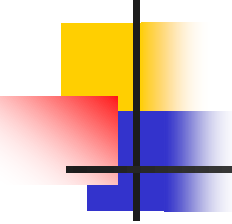
Nonpreemptive Scheduling

- **Once CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.**
- **Virtually all modern OSes use preemptive scheduling algorithms**
 - Windows, macOS, Linux, and UNIX...



Preemptive Scheduling

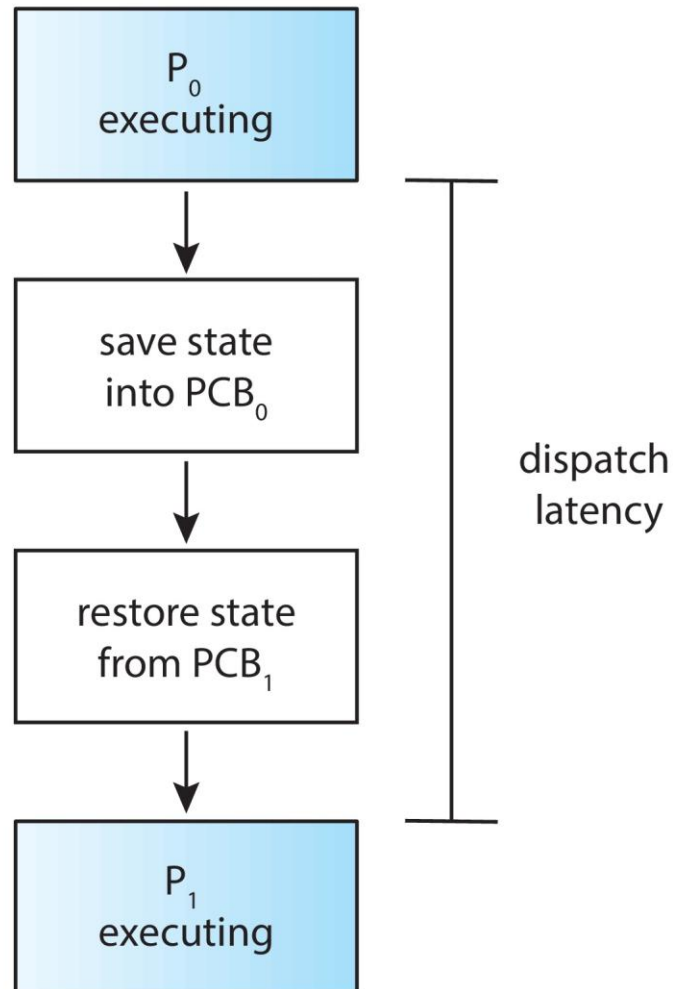
- **Can result in race conditions when accessing to shared data**
 - Need mechanisms (Chapter 6) to coordinate access to shared data, otherwise, accessing shared data would result in an inconsistent state.
- **Affects the design of OS kernel**
 - Consider preemption while in kernel mode
 - kernel support: **not** to preempt a process during the processing of a **system call**
 - ensure the kernel **structures** are consistent
 - Consider interrupts occurring during crucial OS activities
 - code affected by interrupts must be guarded from simultaneous use
 - **disable** INTs at entry and **reenable** INTs at exit

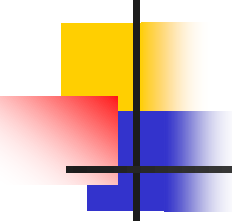


5.1.4 Dispatcher

- The module gives control of the CPU's core to the process selected by the CPU scheduler
- Dispatcher function involves:
 - Switching context
 - Switching to user mode
 - Jumping to the proper location in the user program to restart that program
- ***Dispatch latency*** – time it takes for the dispatcher to stop one process and start another running.

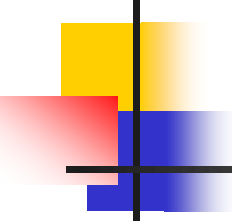
The role of the dispatcher (Fig. 5.3)





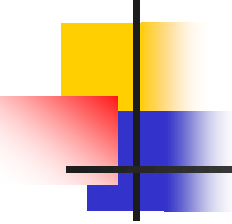
5.2 Scheduling Criteria

- **For comparing CPU scheduling algorithms:**
- ***CPU utilization***
 - We want to keep the CPU as busy as possible
 - how **busy** CPU is
 - e.g., 40% (lightly loaded system) to 90% (heavily loaded system)
- ***Throughput***
 - number of processes completed per time unit
 - what **rate** processes run
 - e.g., 1 process/hour (long processes)
 - e.g., 10 processes/second (short transactions)



Scheduling Criteria (Cont.)

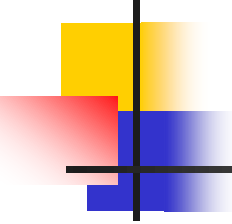
- ★ ■ ***Turnaround time***
 - amount of time to execute a particular process
 - the **interval** from submission to completion
- ★ ■ ***Waiting time***
 - Total time for a process spent waiting in the ready queue
- ***Response time***
 - amount of time from submission of a request until the first response is produced
 - The amount of time it takes to start responding, not the time it takes to output the response



Optimization Criteria

- **Max CPU utilization**
- **Max throughput**
- **Min turnaround time**
- **Min waiting time**
- **Min response time**
- **Min the max response time**
- **Min the variance in the response time**
- **...**





5.3 Scheduling Algorithms

- **5.3.1 First-Come, First-Served Scheduling**
- **5.3.2 Shortest-Job-First Scheduling**
- **5.3.3 Priority Scheduling**
- **5.3.4 Round-Robin Scheduling**
- **5.3.5 Multilevel Queue Scheduling**
- **5.3.6 Multilevel Feedback Queue Scheduling**

This Chapter

- For simplicity of illustration, this chapter consider only one CPU burst (in milliseconds) per process in examples.
- One measure of comparison is the **average waiting time**.

•
•
•

load store
add store
read from file

} CPU burst

wait for I/O

} I/O burst

store increment
index
write to file

} CPU burst

wait for I/O

} I/O burst

load store
add store
read from file

} CPU burst

wait for I/O

} I/O burst

•
•
•

Preemptive Non-preemptive

FCFS		v
SJF	v (SRTF)	v
priority	v	v
RR	v	
Multi-level queue	v	
Multi-level feedback queue	v	



5.3.1 First-Come, First-Served (FCFS) Scheduling

- Example:

Process	Burst Time
---------	------------

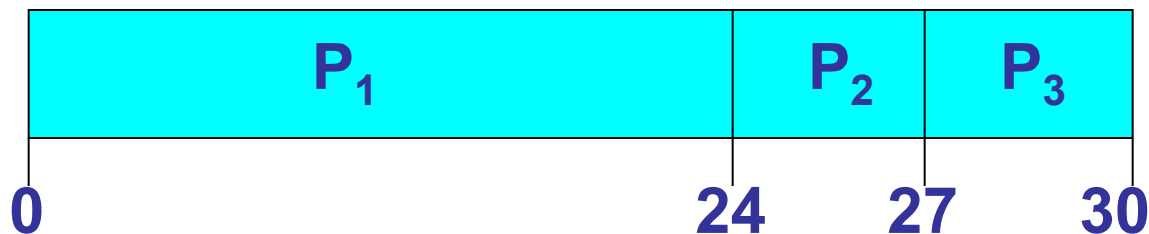
P_1	24
-------	----

P_2	3
-------	---

P_3	3
-------	---

arrival order:
 P_1, P_2, P_3

The Gantt Chart for the schedule is:



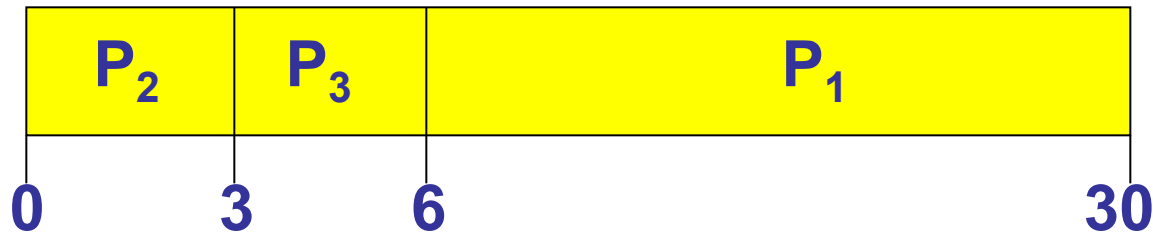
- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

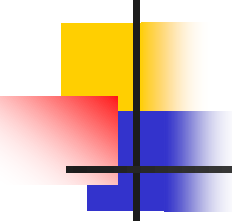
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

Suppose arrival order: P_2, P_3, P_1 .

- The **Gantt chart** for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case.

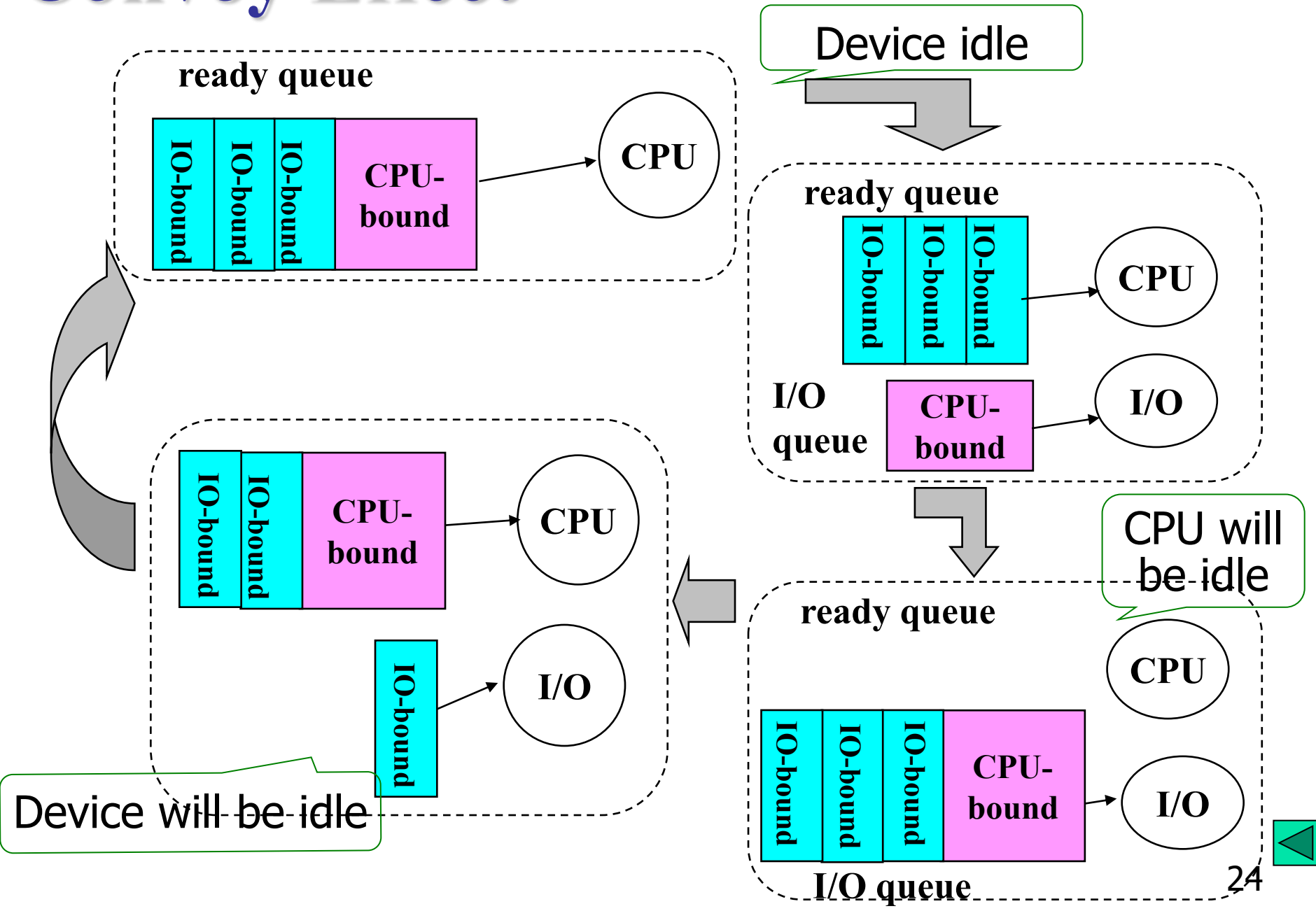


FCFS Scheduling (Cont.)

- The average waiting time is generally not optimal and may vary substantially if the processes' CPU burst times vary greatly.
- Nonpreemptive
 - Troublesome for time-sharing systems
- **convoy effect** 護送效應
 - short process behind long process
 - all the other processes wait for the **one big process** to get off the CPU
 - This effect results in lower CPU and device utilization

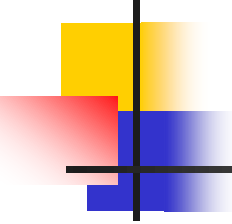
Convoy Effect

assume we have one CPU-bound process
and many I/O-bound processes....



5.3.2 Shortest-Job-First (SJF) Scheduling

- *Shortest-next-CPU-burst* algorithm
- Associate with each process the length of its next CPU burst.
- The process with **shortest next CPU burst** is selected.
 - If equal, processes are scheduled in **FCFS** order.
- ★ ■ Scheduling is done by examining the length of next CPU burst of a process, rather than its total length.



Shortest-Job-First (SJF) Scheduling (Cont.)

- Two schemes:
 - nonpreemptive **SJF**
 - once CPU given to the process it cannot be preempted until completes its CPU burst.
 - preemptive **SJF**
 - if a new process arrives with CPU burst length less than remaining time of current executing process, preempt.
 - Also named **Shortest-Remaining-Time-First (SRTF)**.

Example of Non-Preemptive SJF

Process

P_1

P_2

P_3

P_4

Burst Time

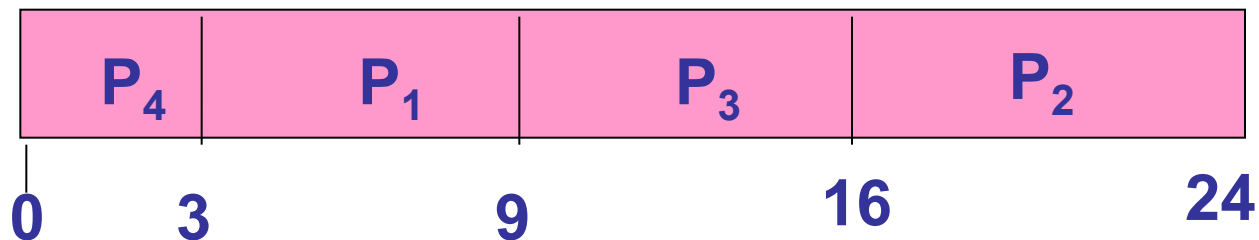
6

8

7

3

- Gantt Chart



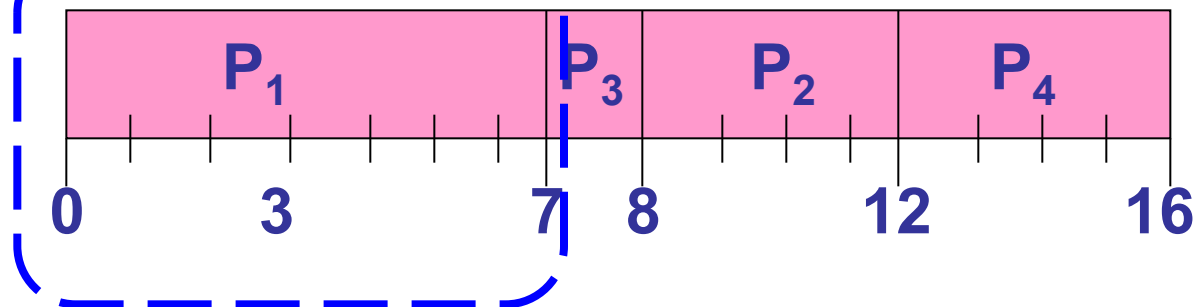
- Average waiting time: $(3 + 16 + 9 + 0)/4 = 7$

Example of Non-Preemptive SJF

- Now we add the concepts of varying arrival times to the analysis

Process	Arrival Time	Burst Time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

■ Gantt Chart



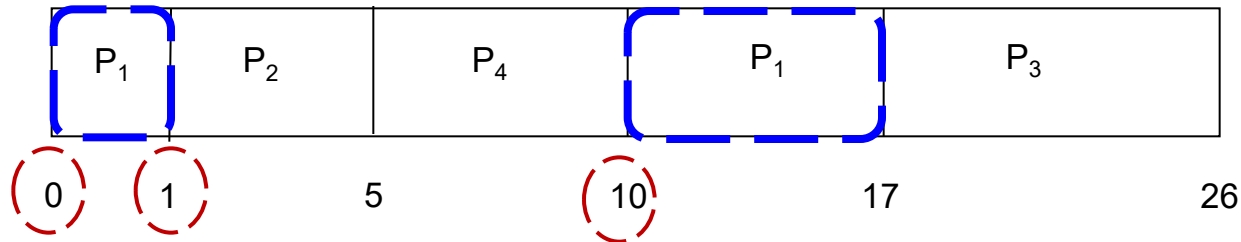
- Average waiting time: $(0 + 6 + 3 + 7)/4 = 4$

Shortest-remaining-time-first

Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

■ *Preemptive SJF Gantt Chart*

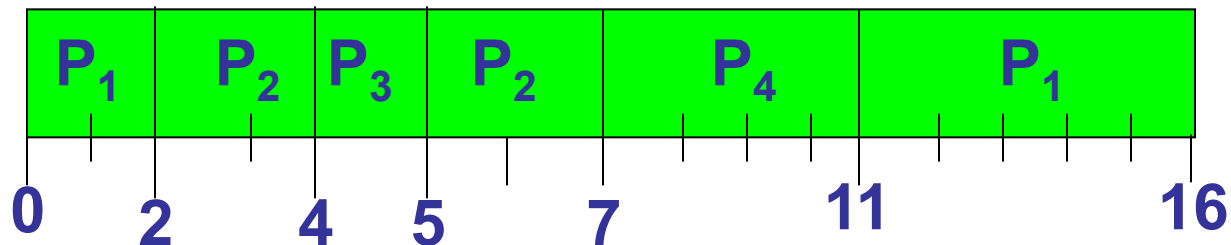


- Average waiting time = $\frac{[(10-1)+(1-1)+(17-2)+(5-3)]}{4} = \frac{26}{4} = 6.5 \text{ msec}$

Example of Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>waiting time</u>
P_1	0	7	
P_2	2	4	
P_3	4	1	
P_4	5	4	

■ Gantt Chart

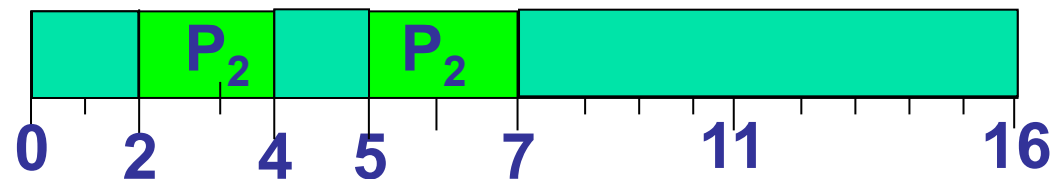


■ Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

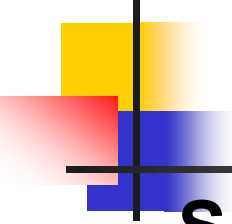
Waiting time

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>waiting time</u>
P_1	0	7	
P_2	2	4	
P_3	4	1	
P_4	5	4	

■ Gantt Chart



■
$$\text{Waiting time} = \text{completion time} - \text{Arrival time} - \text{Execution time}$$



SJF Scheduling

- SJF is provably *optimal*
 - Gives **minimum** average waiting time for a given set of processes
 - Used frequently in long-term scheduling
 - Cannot be implemented in short-term scheduling
- How to know the **length** of the **next CPU burst**?
 - can only predict the length
- **Approximate** SJF scheduling by picking the process with the shortest predicted CPU burst.
- How to **predict** the length of the next CPU burst?

Determining Length of Next CPU Burst

- Predicted as an **exponential average** of the measured lengths of **previous** CPU bursts.

actual length
of n^{th} CPU burst

predicted value
for the next CPU burst

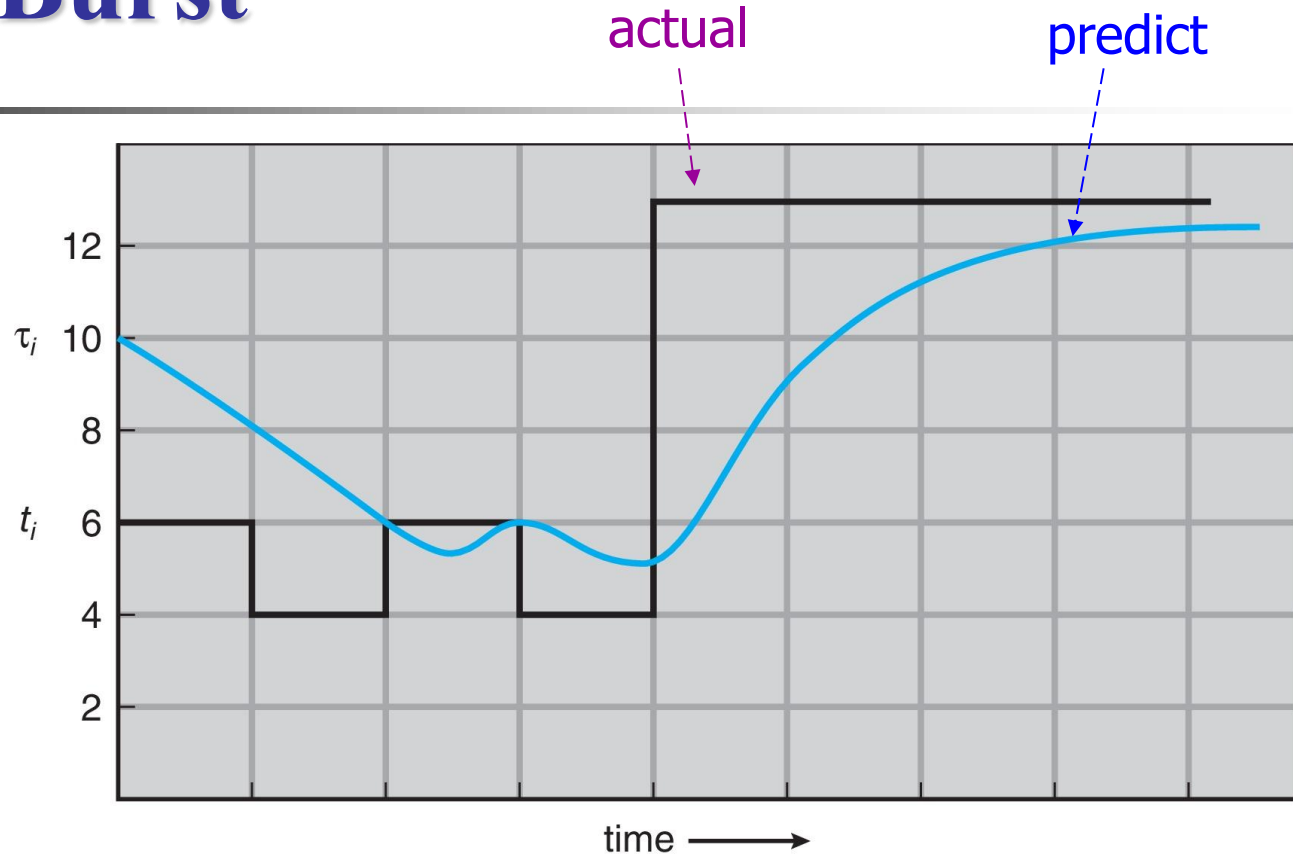
$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

Predicted value

$$\alpha, 0 \leq \alpha \leq 1$$

Prediction of the Length of the Next CPU Burst

- $\alpha = 1/2$
- $\tau_0 = 10$
- t : actual
- τ : predict



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Examples of Exponential Averaging

- $\alpha = 1$

- $\tau_{n+1} = t_n$

Recent history does not count.
Only the actual last CPU burst counts.

- $\alpha = 0$

- $\tau_{n+1} = \tau_n$

- If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor.



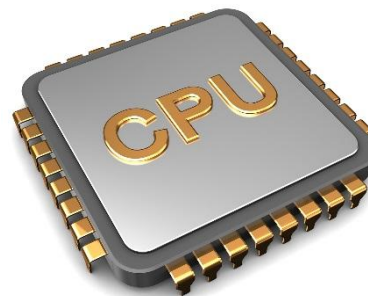
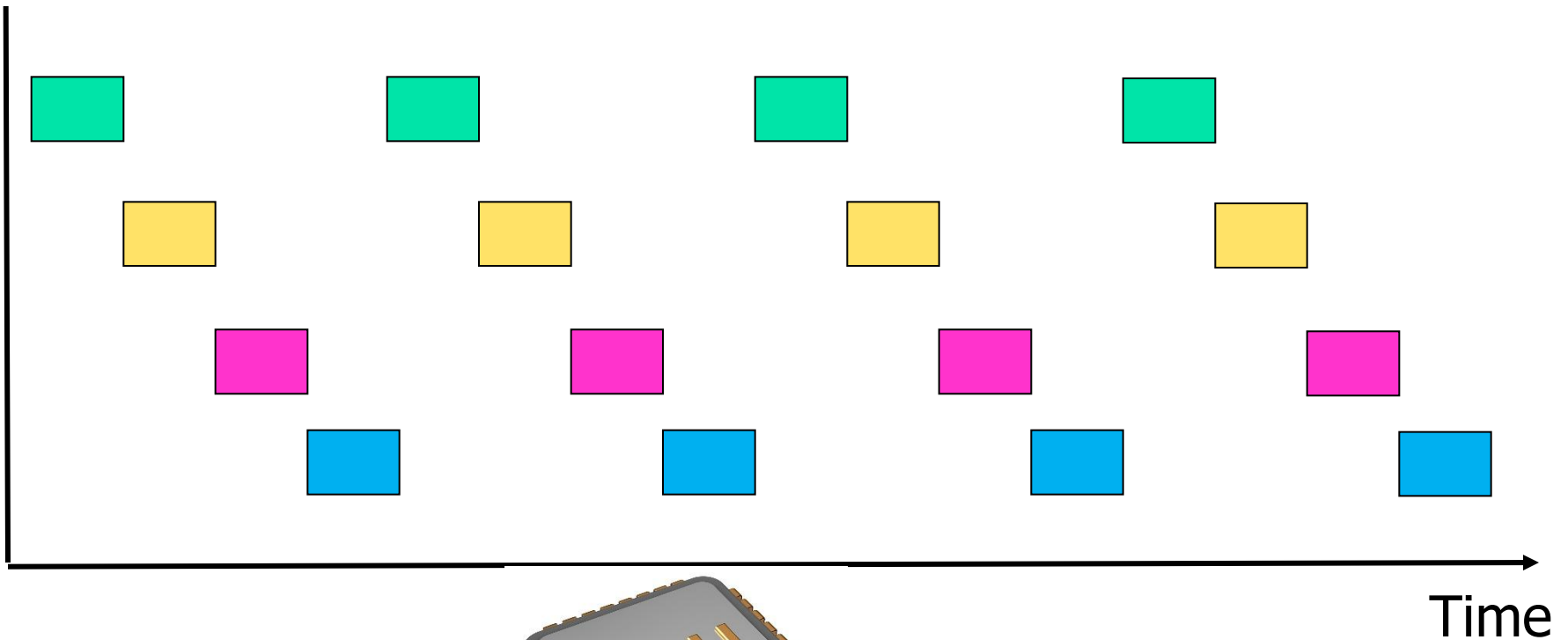
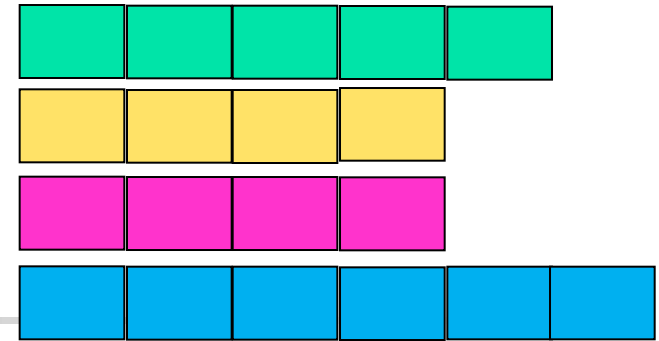
5.3.3 Round-Robin Scheduling (RR)

- Designed especially for **time-sharing systems**.
- Each process gets a small unit of CPU time (**time quantum** or **time slice**), usually 10 to 100 milliseconds.
- When time quantum is expired, the process is preempted. 滿期
- Keep the ready queue as a FIFO queue of processes:
 - new process is added to tail
 - the head or first process is scheduled to run
 - running process is preempted and added to tail if timer goes off



time-sharing systems

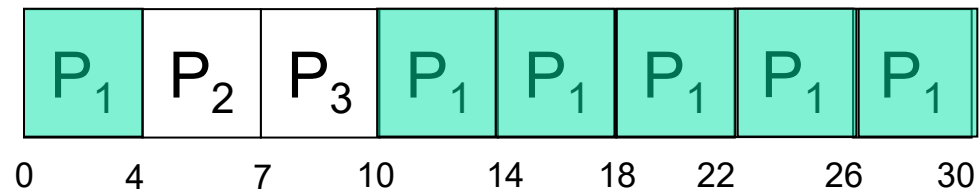
Tasks



Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*.

Example: RR with Time Quantum = 20

<u>Process</u>	<u>Burst Time</u>
----------------	-------------------

P_1	53
-------	----

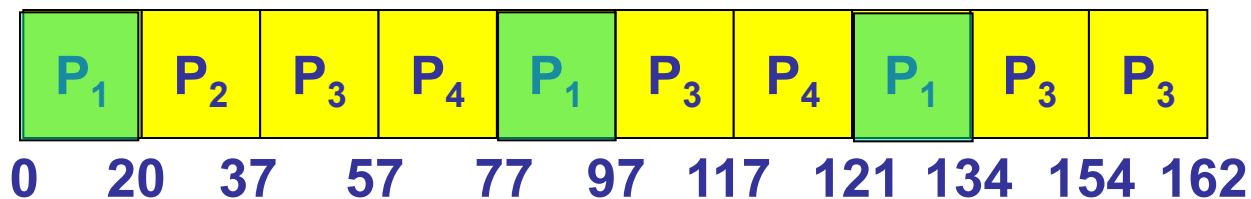
P_2	17
-------	----

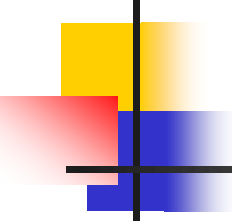
P_3	68
-------	----

P_4	24
-------	----

Assume all arrived
at time 0

- The Gantt chart is:



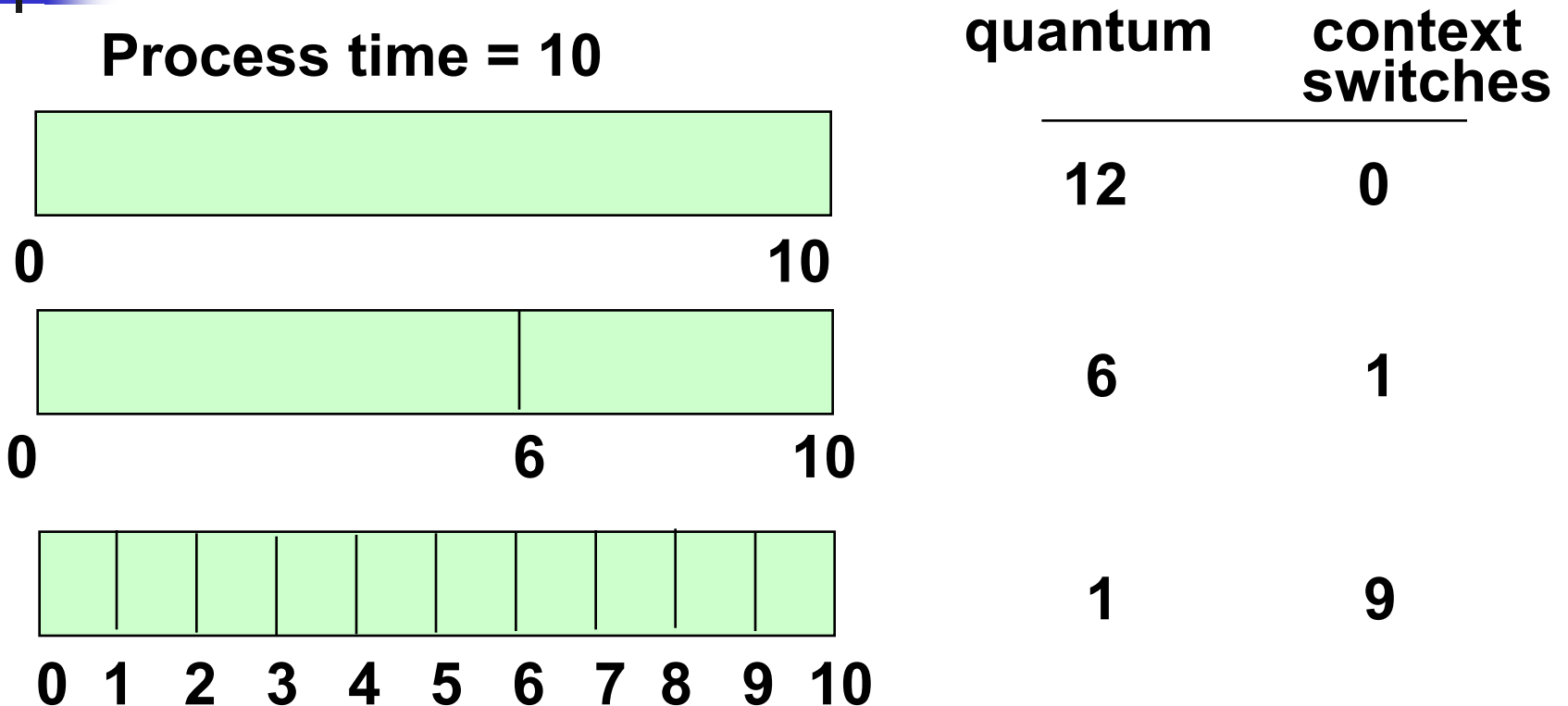


Round Robin (Cont.)

- **n processes in ready queue and q time quantum: each process**
 - gets one q CPU time every nq time units, that is, gets $1/n$ CPU every q time units
 - waits no longer than $(n-1)*q$ time units
- **Performance of RR depends on q**
 - if $q \rightarrow \infty$, $RR \Rightarrow FCFS$
 - if $q \rightarrow$ smaller q , more times of context switches
- **Time quantum must be large with respect to context-switch time, it should not be too large.**

就...而言

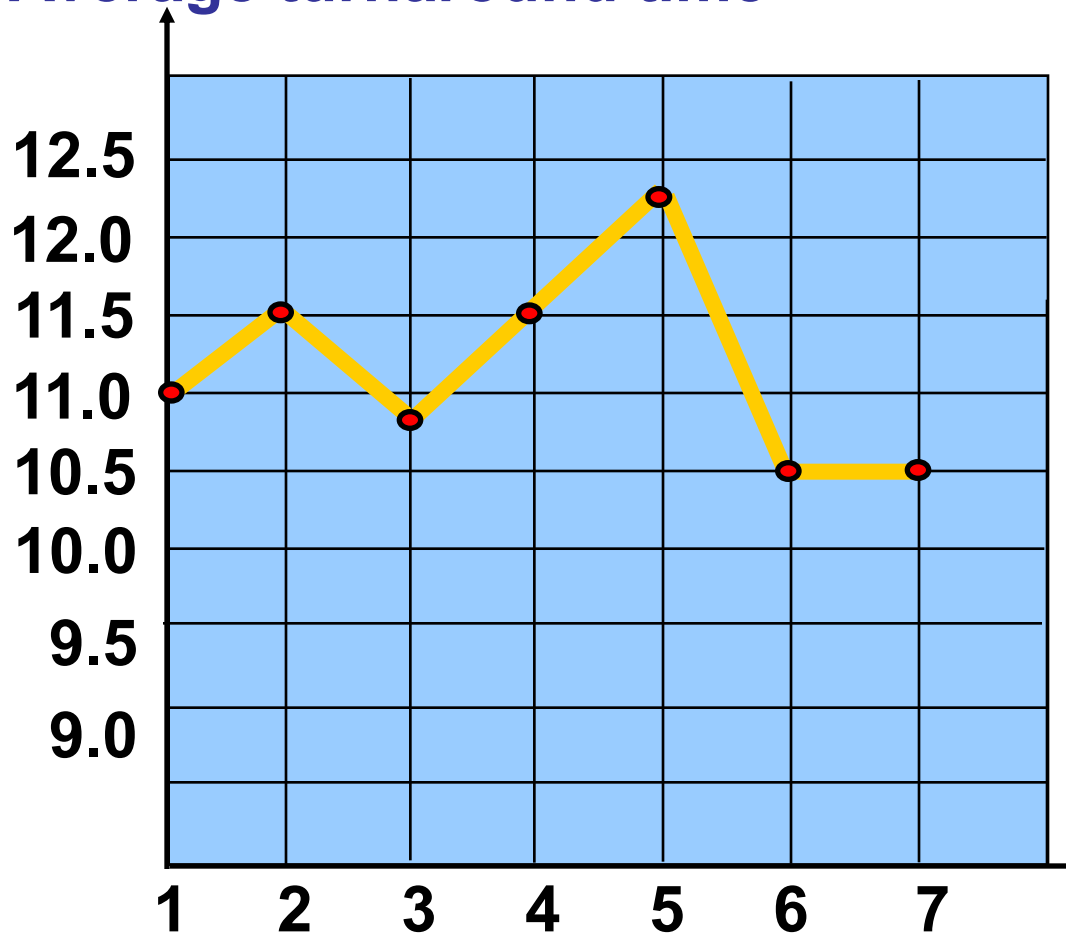
How a Smaller Time Quantum Increases Context Switches



- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec

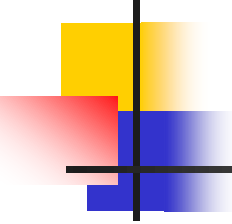
Turnaround Time Varies With The Time Quantum

Average turnaround time



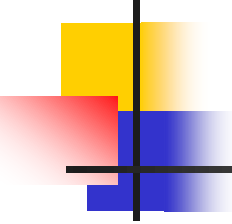
Process	time
P1	6
P2	3
P3	1
P4	7





5.3.4 Priority Scheduling

- A **priority number** (integer) is associated with each process
- Process with the **highest priority** is selected first
- **Equal-priority** processes are scheduled in **FCFS** order.
- Priority can be either internally or externally defined
 - **External** priority: important, political...
 - **Internal** priority: measurable quantities... time limits....
- **SJF is a priority algorithm where**
 - priority is the predicted next CPU burst time
 - the **larger** the CPU burst, the **lower** the priority



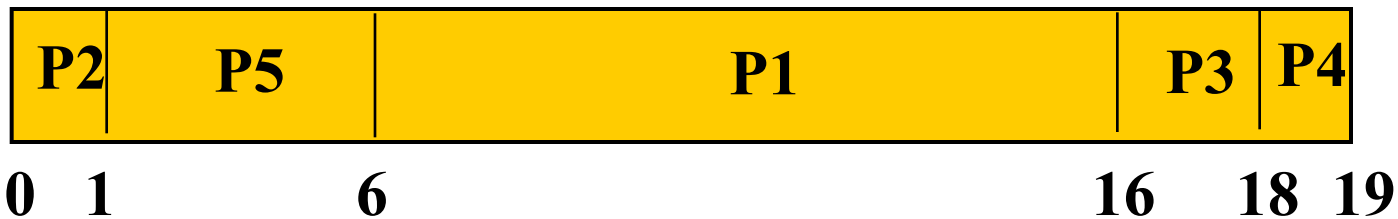
Priority Scheduling (Cont.)

- Can be either **preemptive** or **nonpreemptive**
- **Preemptive** priority scheduling:
 - preempt CPU if newly arrived process has **higher** priority than that of currently running process
- **Nonpreemptive** priority scheduling:
 - put the new process at the head of ready queue

Example of Non-Preemptive Priority Scheduling

- Assume all arrived at time 0, in the order p1,p2,...,p5
- Assume low numbers represent high priority

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2



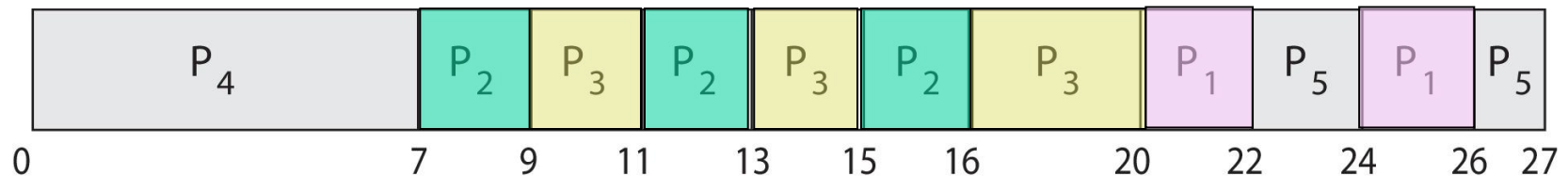
- Average waiting time = 8.2 milliseconds

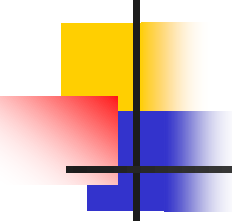
Priority Scheduling with Round-Robin

2 ms time quantum

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

- Run the process with the highest priority. Processes with the same priority run round-robin





Priority Scheduling (Cont.)

- Major problem $\equiv$ **indefinite blocking** (or **starvation**) of low-priority processes
 - low priority processes may never execute
- A process that is ready to run but waiting for the CPU can be considered **blocked**.
- Solution $\equiv$ **Aging**
 - gradually increasing the priority of processes that wait in the system for a long time



5.3.5 Multilevel Queue Scheduling

- **Classified** processes into different groups, since different groups might have different scheduling needs.
 - e.g., foreground (interactive) processes
background (batch) processes
- **Partitioned** ready queue into separate queues
 - processes are permanently assigned to one queue
 - each queue has its own scheduling algorithm
 - e.g., foreground – RR
background – FCFS

Separate queues for each priority (Fig. 5.7)

priority = 0

T_0	T_1	T_2	T_3	T_4
-------	-------	-------	-------	-------

priority = 1

T_5	T_6	T_7
-------	-------	-------

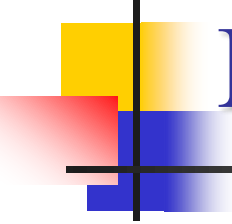
priority = 2

T_8	T_9	T_{10}	T_{11}
-------	-------	----------	----------



priority = n

T_x	T_y	T_z
-------	-------	-------



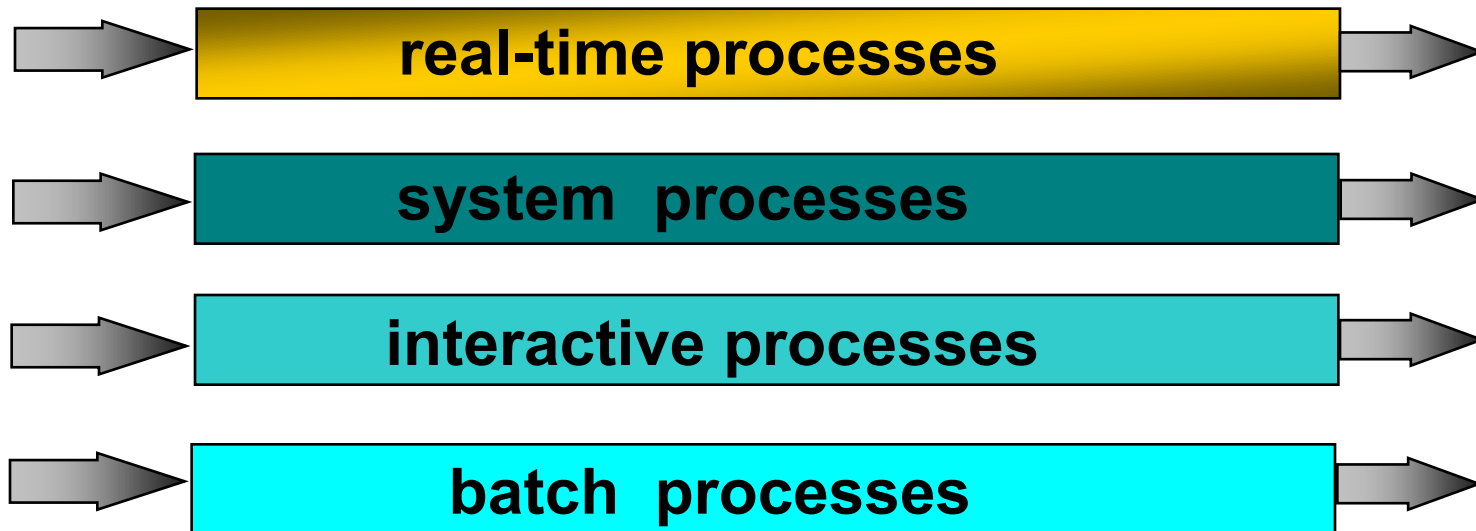
Multilevel Queue (Cont.)

- **Scheduling among the queues (two possible ways):**
 - **fixed-priority preemptive scheduling**
 - each queue has **absolute priority** over lower-priority queues
 - no processes could run unless all higher-priority queues are **empty**
 - a process is **preempted** while a new higher-priority process **entered**
 - possibility of starvation
 - **time slice among the queues**
 - each queue gets a certain portion of CPU time which it can schedule among its processes
 - i.e., 80% to foreground in RR
20% to background in FCFS

Example of Multilevel Queue Scheduling

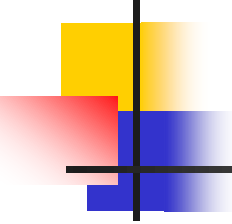
In the case of **fixed-priority preemptive scheduling**, no process in batch queue can be run unless all higher-priority queues are empty

Highest priority



Lowest priority





5.3.6 Multilevel Feedback Queue Scheduling

- The most general CPU scheduling algorithm and the most complex one
- **A process can be moved between the various queues (no permanent, fixed priority of a process)**
 - Separate processes with different CPU-burst characteristics.
- **Aging prevents starvation**
 - **uses too much CPU time**, process is moved to a lower-priority queue
 - **waits too long in queue**, process is moved to a higher-priority queue

Example of Multilevel Feedback Queue

- **Three queues:**

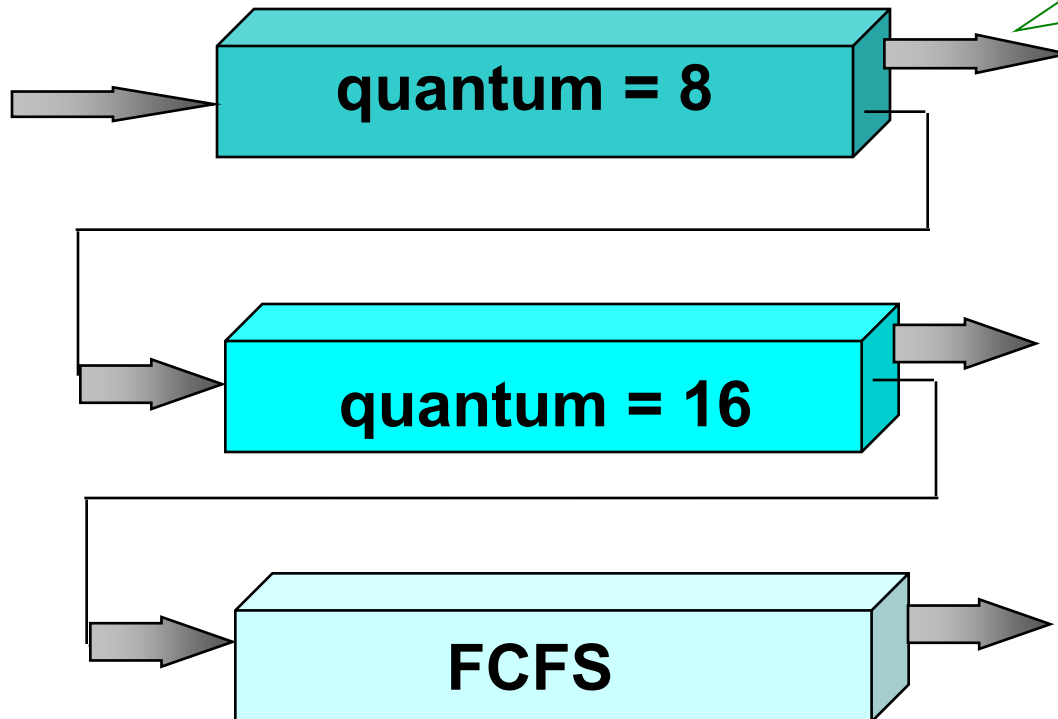
- Q_0 – time quantum 8 milliseconds
- Q_1 – time quantum 16 milliseconds
- Q_2 – FCFS

- **Scheduling**

- A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
- At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .

Example of Multilevel Feedback Queues

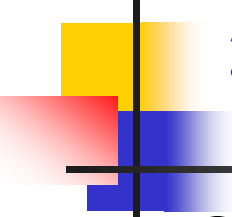
Gives the highest priority to any process with a CPU burst of 8 ms or less.



Multilevel Feedback Queue (Cont.)

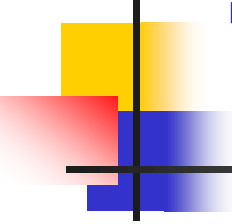
- **A multilevel-feedback-queue scheduler is defined by the following parameters:**
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service





5.4 Thread Scheduling

- On OS supporting user-level and kernel-level threads, it is kernel-level threads that are being scheduled by OS.
- To run on a CPU, user-level threads must ultimately be mapped to an associated kernel-level thread.
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on an available LWP
 - Known as **process-contention scope (PCS)** since competition for CPU is among threads belonging to the same process
- Kernel thread scheduled onto available CPU is **system-contention scope (SCS)** – competition for CPU is among all threads in system



Thread Scheduling (Cont.)

- **System using one-to-one model, (Windows, Linux, and Solaris), schedule threads using only SCS.**
- **POSIX Pthread API allows specifying either PCS or SCS during thread creation**
 - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
 - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling.
- **Can be limited by OS – Linux and Mac OS X systems allow only PTHREAD_SCOPE_SYSTEM**

Pthread Scheduling API (Fig. 5.10)

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
    int i;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* first inquire on the current scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("PTHREAD_SCOPE_PROCESS");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD_SCOPE_SYSTEM");
        else
            fprintf(stderr, "Illegal scope value.\n");
    }
}
```

POSIX Pthread API allows specifying either **PCS** or **SCS**

many-to-many

many-to-one

one-to-one

Pthread Scheduling API (Fig. 5.10)

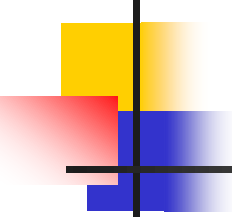
```
/* set the scheduling algorithm to PROCESS or SYSTEM */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

/* create the threads */
for (i = 0; i < NUM THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);

/* now join on each thread */
for (i = 0; i < NUM THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    printf("I am a thread\n");
    pthread_exit(0);
}
```





5.5 Multiple-Processor Scheduling

- CPU scheduling becomes more complex when multiple CPUs are available.
- **Load sharing** becomes possible if multiple CPUs are available.
- **Homogeneous** and **Heterogeneous** systems:

同質性 ■ **homogeneous processors**: processors are identical in their functionality

- we can then use any available processor to run any process in the queue

異質性 ■ **heterogeneous processors**: processors are different in their capabilities

5.5.1 Approaches to Multiple-Processor Scheduling

非對稱式

■ Asymmetric multiprocessing

- All system activities are handled by one specific processor – the master server. Other processors only execute user code.
 - Only one processor accesses the system data structures, reducing the need for data sharing.

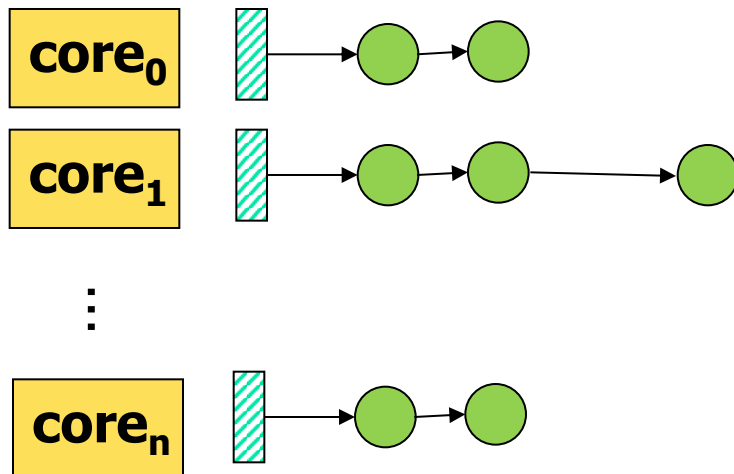
對稱式

■ Symmetric multiprocessing (SMP)

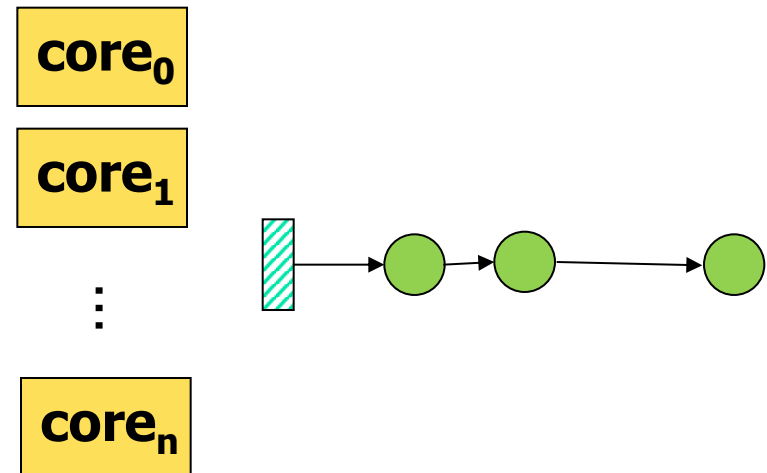
- Each processor is self-scheduling.
- All processes may be in a common ready queue, or each processor may have its own private queue of ready processes.
- Scheduling proceeds by having the scheduler for each processor examine the ready queue and select a process to execute.
- Virtually all modern OSes support SMP
 - e.g., Windows, Linux, macOS, Android, iOS

Organization of ready queues

- each processor has its own private ready queue

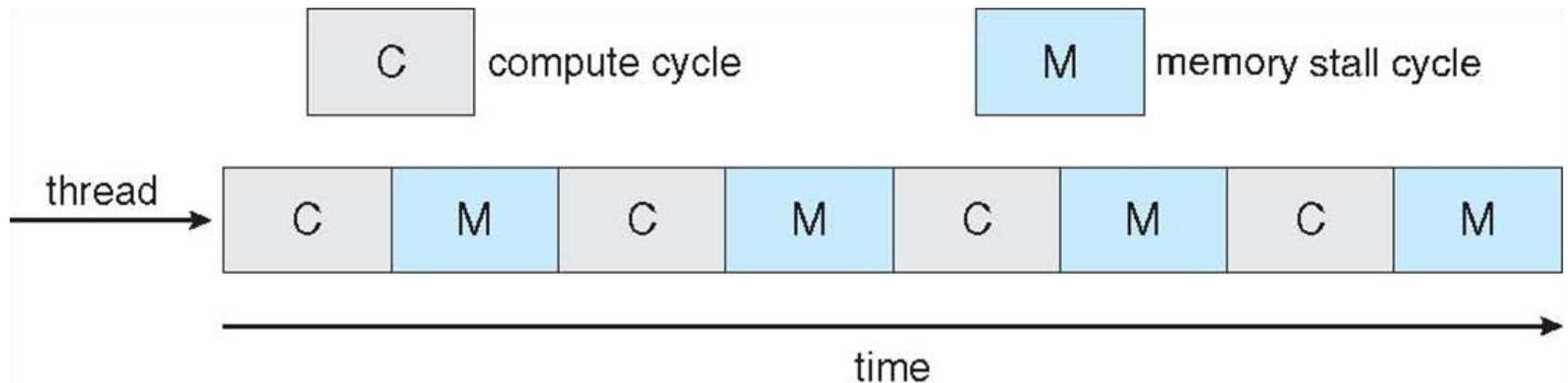


- a common ready queue



5.5.2 Multicore Processors

- **Multicore processors**: recent trend to place multiple processor cores on same physical chip
 - Appears to OS to be a separate physical processor
 - Faster and consume less power than each processor has its own physical chip
- **Memory stall**
 - When a processor accesses memory, it spends a significant amount of time waiting for the data to become available.
 - May occur such as a cache miss

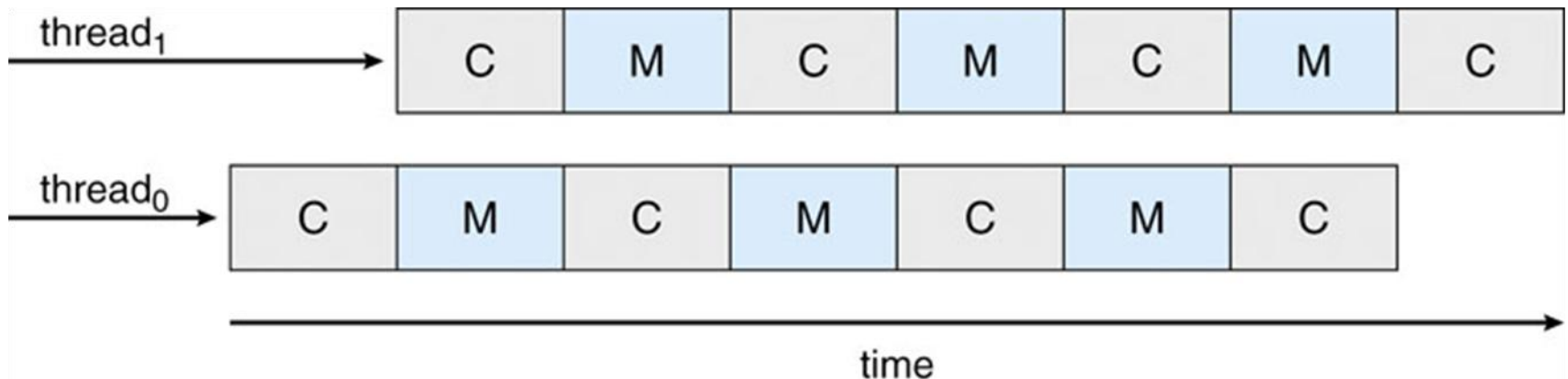


Multithreaded multicore system

(Fig. 5.13)

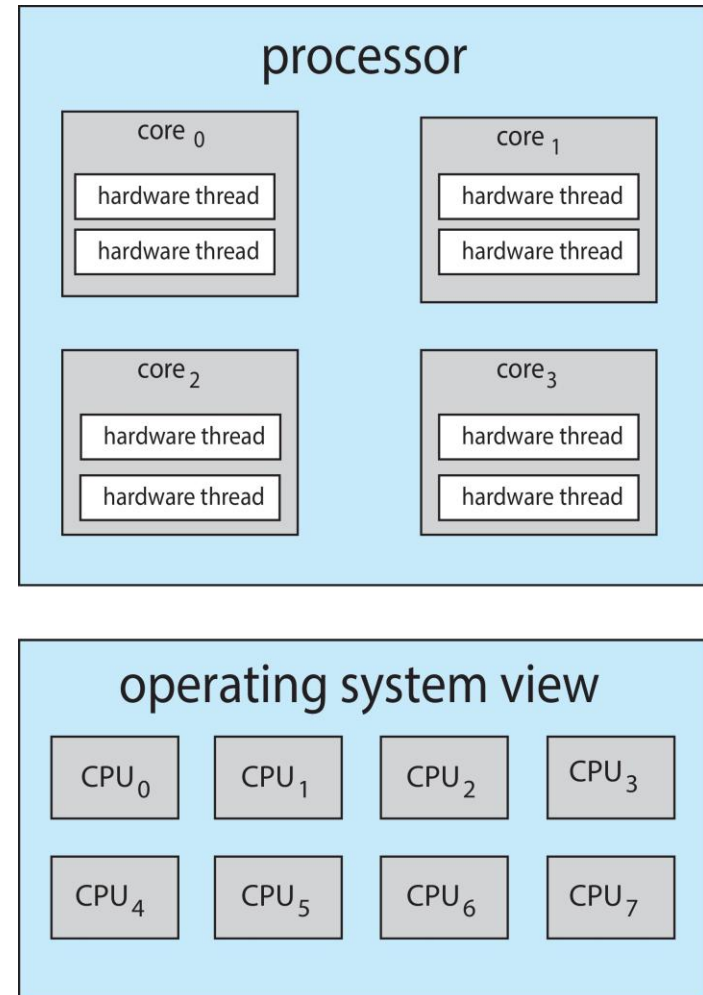
- **Multiple threads per core also growing**

- Multithreaded processor cores in which two or more hardware threads are assigned to each core
- Takes advantage of memory stall to make progress on another thread while memory retrieve happens
- For OS, each hardware thread appears as a logical processor that is available to run a software thread
- On a dual-threaded, dual-core system, four logical processors are presented to OS



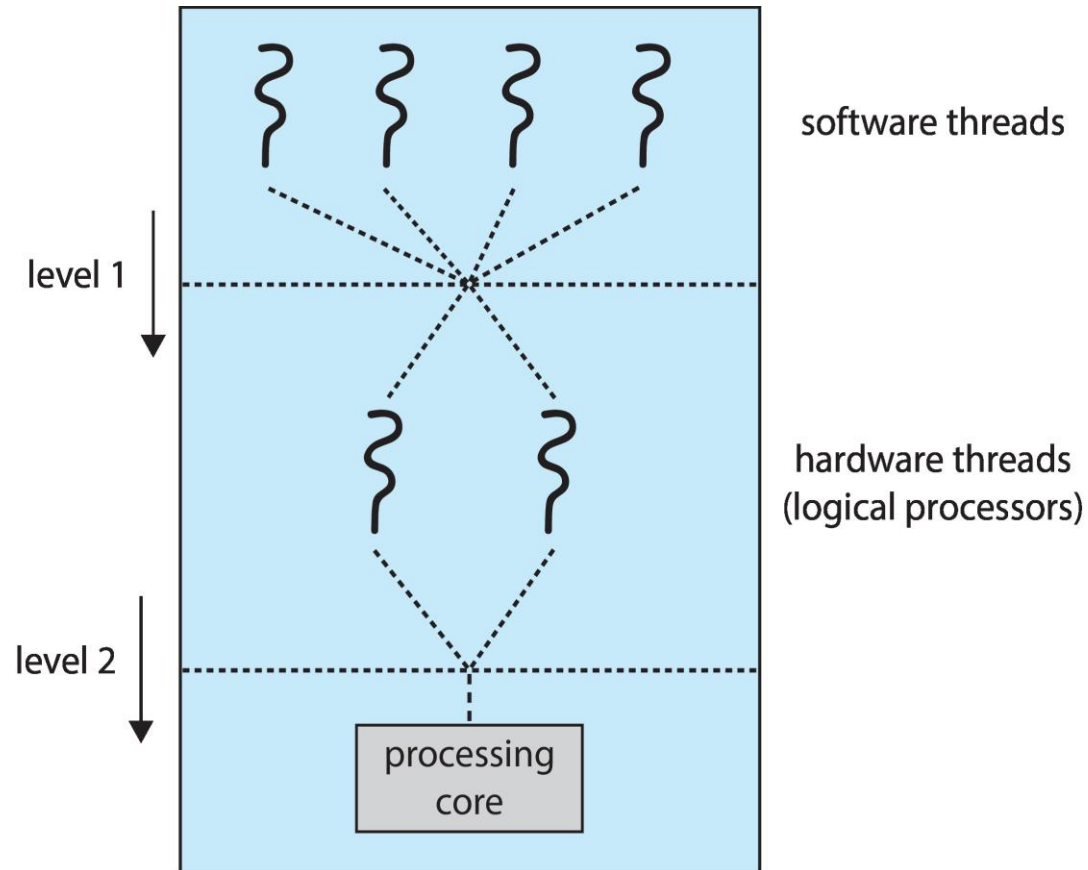
Multithreaded Multicore System

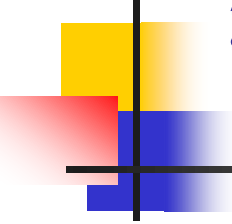
- **Chip-multithreading (CMT)** assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)
- On a quad-core system with 2 hardware threads per core, OS sees 8 logical processors.



Multithreaded Multicore System

- If OS is aware of the level of processor resource sharing, it can schedule software threads onto logical processors that do not share resources.



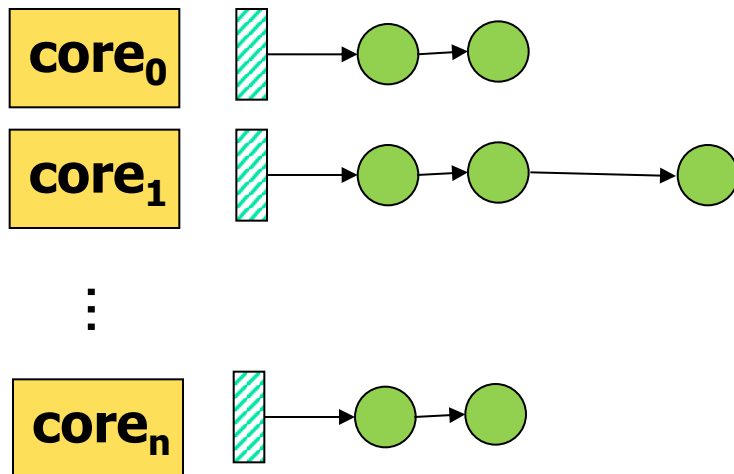


5.5.3 Load Balancing

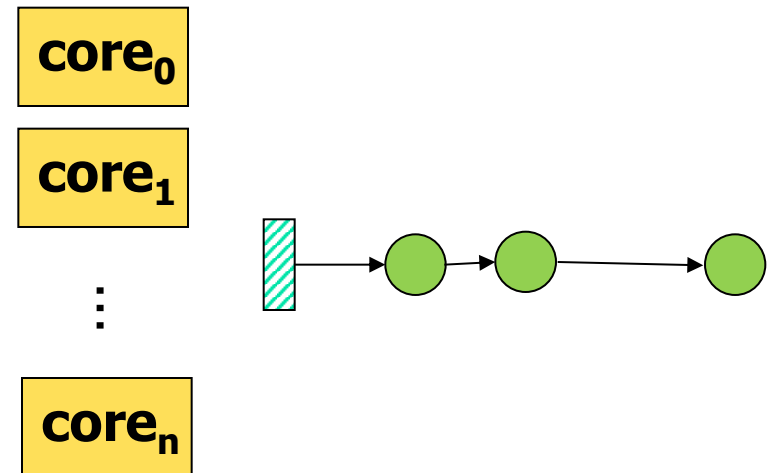
- ***Load balancing*** – attempts to keep the workload evenly distributed across all processors in an SMP system.
 - To fully utilize the benefits to having more than one processor.
 - Typically only necessary on systems where each processor has its own private queue of eligible processes to execute.
 - Most OSes use this way
- On systems with a common run queue, load balancing is often unnecessary
 - Because once a processor becomes idle, it immediately extracts a runnable process from the common run queue.

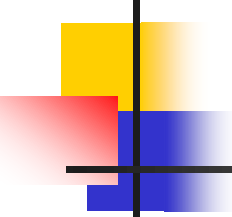
Load Balancing

- each processor has its own private ready queue



- a common run queue





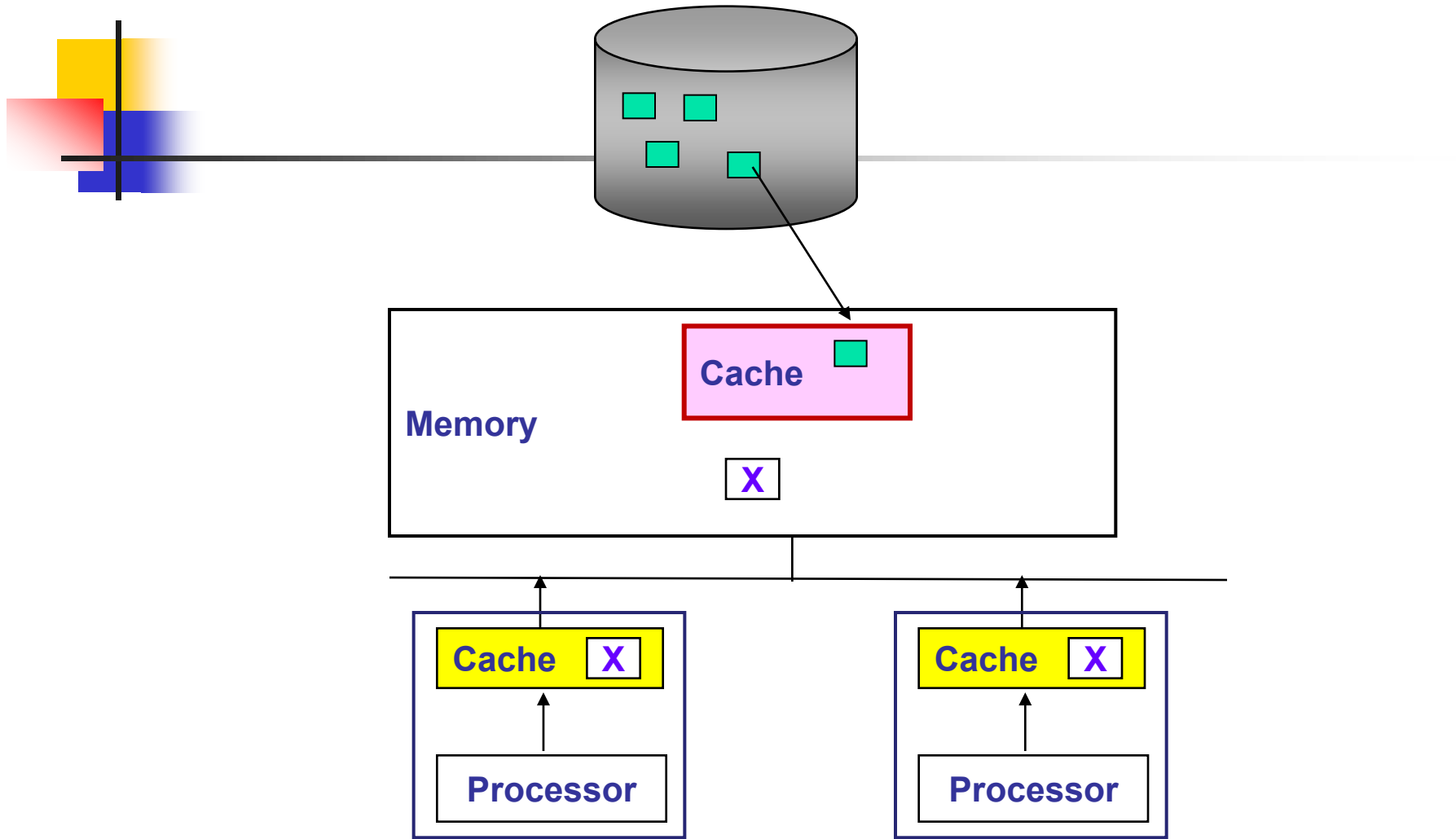
Load Balancing (Cont.)

- **Two general approaches to load balancing:**
 - **Push migration**
 - a specific task periodically checks the load on each processor, if it finds an imbalance, evenly distributes the load by moving (or pushing) processes from overloaded to idle or less-busy processors
 - **Pull migration**
 - occurs when an idle processor pulls a waiting task from a busy processor
- **Push and pull migration** are often implemented in parallel on load-balancing systems.
 - e.g., Linux scheduler and FreeBSD ULE scheduler

喜好, 傾向

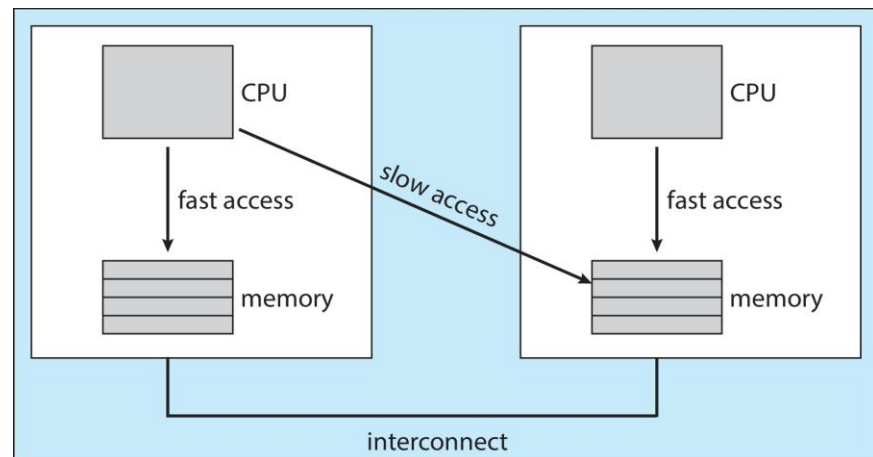
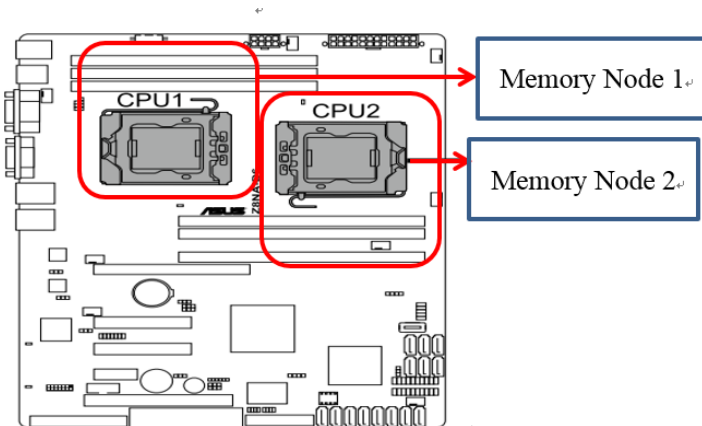
5.5.4 Processor Affinity

- **Processor affinity** – a process has an affinity for the processor on which it is currently running.
 - Most SMP systems attempt to keep a process running on the same processor, to take advantage of its data being in that processor's **cache memory**
- **Processor affinity takes several forms:**
 - **soft affinity**
 - Attempt to keep a process running on the same processor, but not guaranteeing that it will do so
 - **hard affinity**
 - Some systems provide system calls that allow a process to specify a subset of processors on which it may run.
 - Many systems provide both soft and hard affinity
 - e.g., Linux provides `sched_setaffinity()` system call to support hard affinity

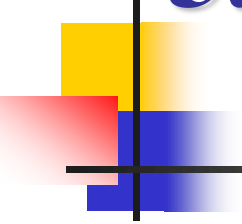


NUMA and CPU Scheduling

- An architecture featuring **non-uniform memory access (NUMA)**
 - A CPU has faster access to some parts of main memory than to other parts
 - CPU scheduling and memory-placement algorithms work together to consider affinity
 - If the OS is **NUMA-aware**, it will assign memory close to the CPU where the thread is running on.



5.6 Real-Time CPU Scheduling



- **CPU scheduling for real-time OSes involves special issues.**
- **Soft real-time systems**
 - No guarantee as to when a critical real-time process will be scheduled
 - Guarantee only that the process will be given preference over noncritical processes
- **Hard real-time systems**
 - A task must be serviced by its deadline; service after the deadline has expired is the same as no service at all.

1.10.6 Real-Time Embedded Systems

嵌入式系統

- An ***embedded system*** is a combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform specific tasks.
- Embedded systems vary considerably. 相當大的
 - Some have OSes (general-purpose computer running standard OS with special-purpose applications, limited features)
 - Some perform tasks without an OS
- Embedded systems almost always run embedded real-time OS.
 - A **real-time system** is used when rigid time requirements have been placed on the operation of a processor or the flow of data

即時

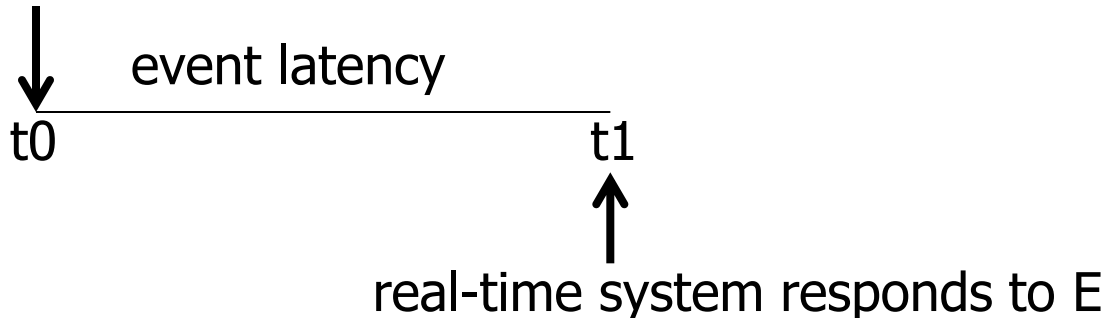
Real-Time Systems

- **Often used as a control device in a dedicated application**
 - e.g., controlling scientific experiments, medical imaging systems, industrial control systems, and some display systems, automobile-engine, home-appliance controllers, weapon systems ...
- **Have well-defined, fixed time constraints.**
- **Processing *must* be done within the defined constraint, or the system will fail.**
- **A real-time system functions correctly only if it returns the correct result within its time constraints.**

5.6.1 Minimizing Latency

- Consider the event-driven nature of a real-time system
 - The system is typically waiting for an event in real time to occur
 - The system must respond to and service it as quickly as possible.
- **Event latency**
 - The amount of time that elapses from when an event occurs to when it is serviced.

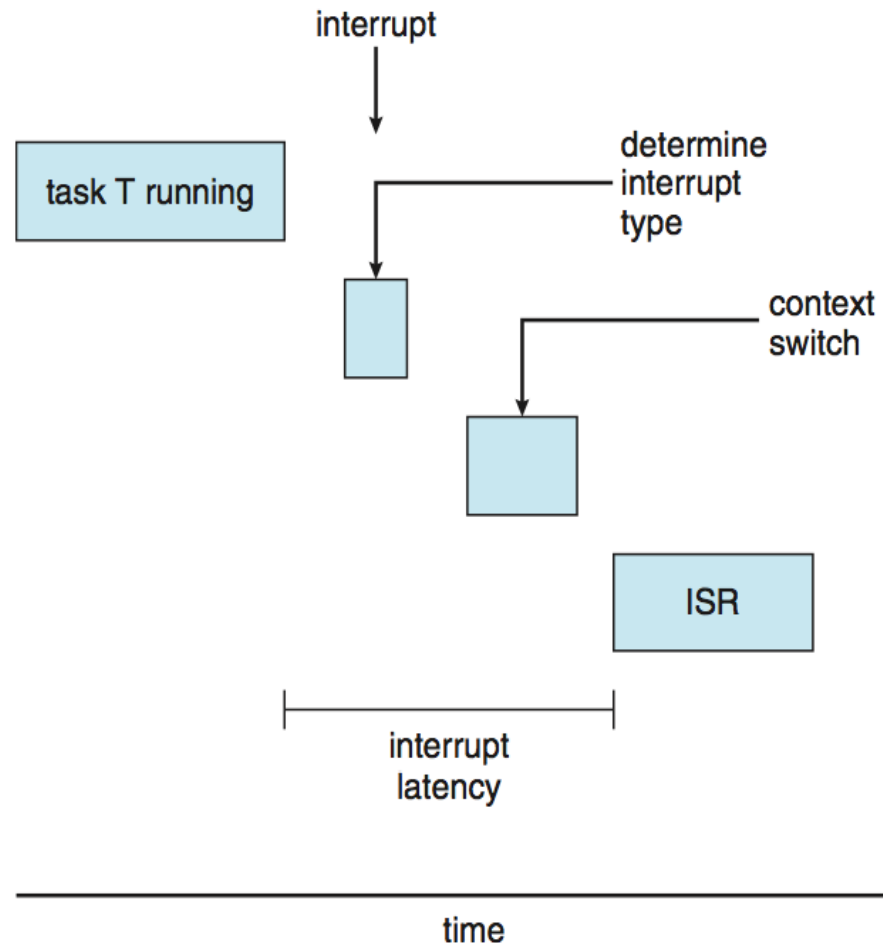
event E first occurs



Two types of latencies affect performance

■ Interrupt latency

- time from arrival of an interrupt at the CPU to the start of the routine that services the interrupt
- Real-time OSes should minimize interrupt latency
- Should be bounded to meet strict requirements for hard real-time systems



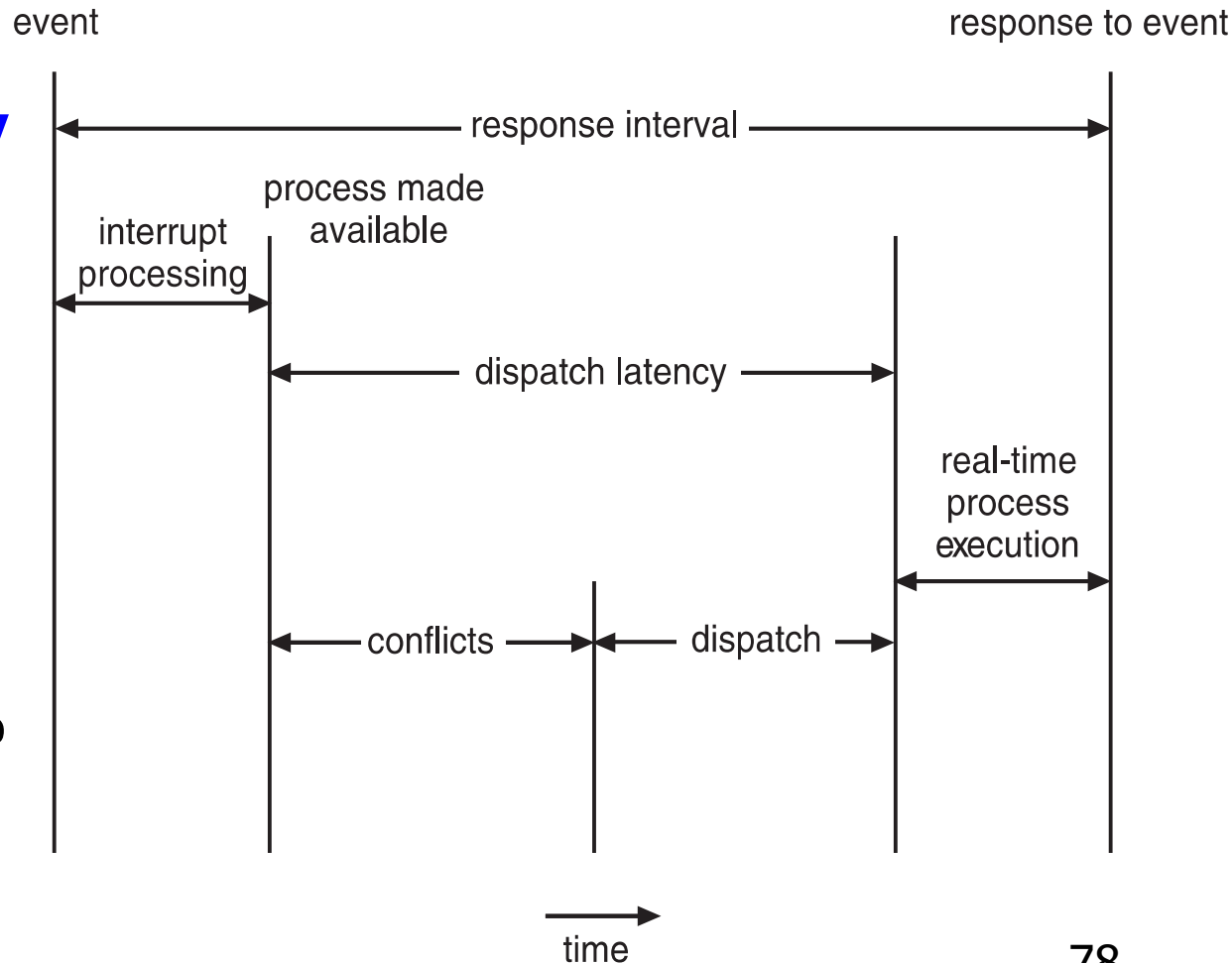
Two types of latencies affect performance (Cont.)

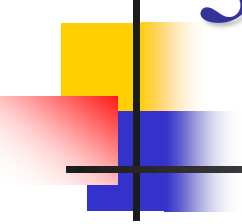


■ Dispatch latency

- time for schedule to take current process off CPU and switch to another

- The most effective technique for keeping dispatch latency low is to provide preemptive kernels





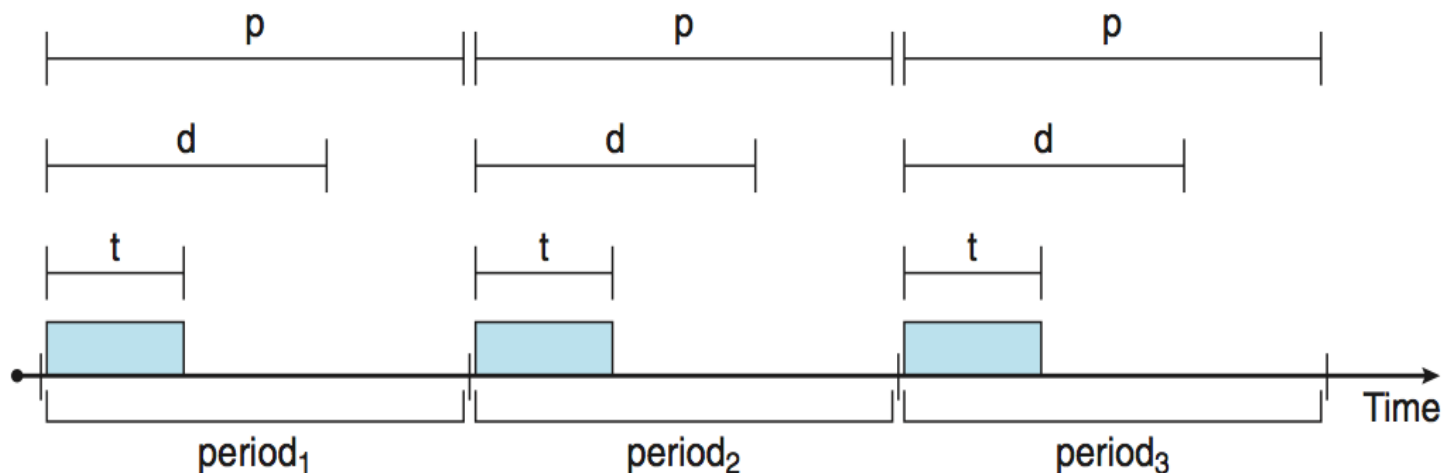
5.6.2 Priority-based Scheduling

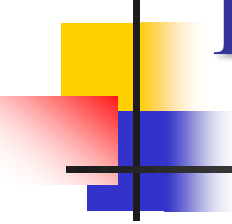
- **For real-time scheduling, scheduler must support preemptive, priority-based scheduling**
 - But only guarantees soft real-time
 - e.g., Linux, Windows, Solaris assign real-time processes the highest priority
- **For hard real-time must also provide ability to meet deadlines**

Priority-based Scheduling (Cont.)

週期性的

- **Processes have new characteristics: periodic**
 - Processes require the CPU at constant intervals
 - Has a fixed **processing time** t , a **deadline** d , a **period** p
 - $0 \leq t \leq d \leq p$
 - **Rate** of periodic task is $1/p$



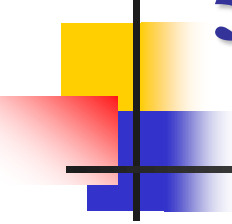


Priority-based Scheduling (Cont.)

- A process may have to announce its deadline requirements to the scheduler.

准入控制

- Using **admission-control algorithm**
 - The scheduler either admits the process, guaranteeing that the process will complete on time, or rejects the request as impossible if it cannot guarantee that the task will be serviced by its deadline.



5.6.3 Rate Monotonic Scheduling

速率

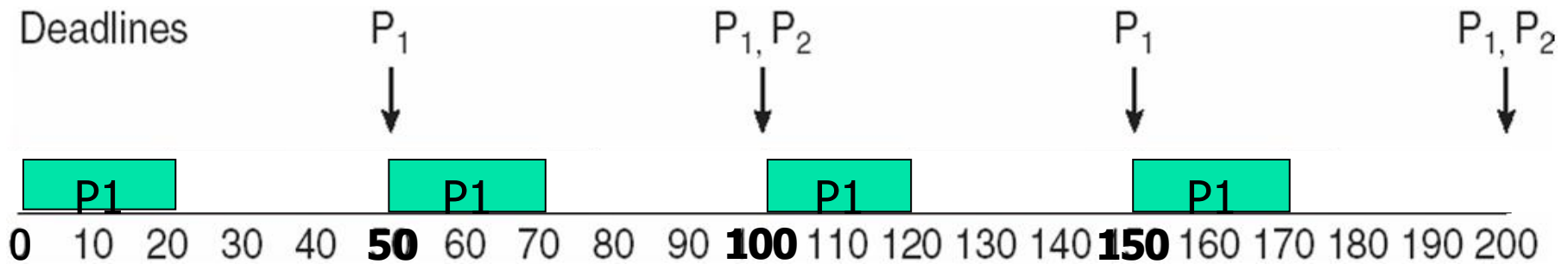
單調的

- Schedules periodic tasks using a static priority policy with preemption
- Each periodic task is assigned a priority inversely based on its **period**
 - Shorter periods = higher priority
 - Longer periods = lower priority
 - Assign a higher priority to tasks that require the CPU more often
 - Assume the processing time of a periodic process is the same for each CPU burst

週期愈短的先作
速率愈高的先作

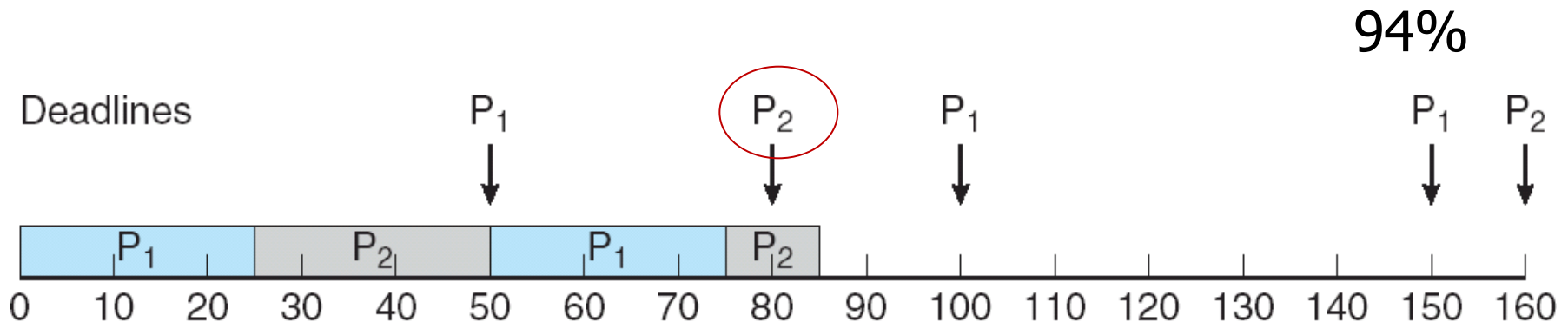
Rate Monotonic Scheduling (Cont.)

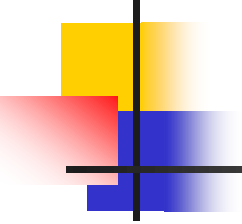
- e.g., two processes **P1: $p_1=50, t_1=20$ P2: $p_2=100, t_2=35$**
- **Admission control:**
 - Check whether it is possible to schedule these tasks to meet their deadlines
 - CPU utilization: $P1=20/50=0.4$ $P2=35/100=0.35$
- **P_1 is assigned a higher priority than P_2 .**



Missed Deadlines with Rate Monotonic Scheduling

- Rate monotonic is optimal, but has a limitation: CPU utilization is bounded, and it is not always possible fully to maximize CPU resources.
- Worst-case CPU utilization for scheduling N processes:
 - $N(2^{1/N} - 1)$ $N=2$, bounded at 83%
- e.g., two processes **P1: p1=50, t1=25** **P2: p2=80, t2=35**



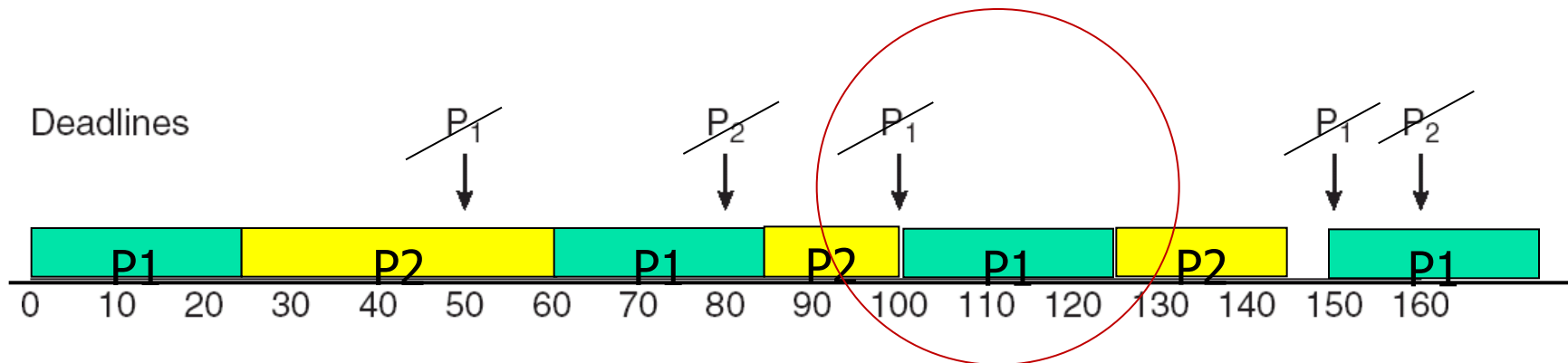


$$N(2^{1/N} - 1)$$

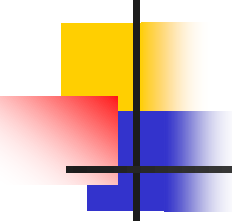
$$2 \times (\sqrt{2} - 1) =$$
$$0.82842712474$$

5.6.4 Earliest Deadline First Scheduling (EDF)

- Process announce its deadline to scheduler when it becomes runnable
- **Dynamically** assigns priorities according to deadline:
 - the earlier the deadline, the higher the priority
 - the later the deadline, the lower the priority
- e.g., two processes **P1: $p_1=50$, $t_1=25$** **P2: $p_2=80$, $t_2=35$**



Earliest Deadline First Scheduling (EDF) (cont.)



- Unlike rate-monotonic algorithm, EDF does not require processes be periodic, nor must a process require a constant amount of CPU time per burst.
- EDF is theoretically optimal
 - Can schedule processes so that each process can meet its deadline and CPU utilization will be 100%
 - Impossible to achieve 100%: cost of context switching between processes and interrupt handling

5.6.5 Proportional Share Scheduling

成比例的

- Operate by allocating T shares among all applications.
- An application receives N shares of time, thus ensuring that the application will have N / T of the total processor time, where $N < T$
- e.g., $T=100$ shares (A 50 shares, B 15 shares, C 20 shares)
- Must work with an admission-control policy
 - Admit a client requesting a particular number of shares only if sufficient shares are available

5.6.6 POSIX Real-Time Scheduling

- The POSIX.1b standard
- POSIX API provides functions related to scheduling real-time threads
- Defines two scheduling classes for real-time threads:
 - **SCHED_FIFO** - threads are scheduled using FCFS with a FIFO queue. There is no time-slicing for threads of equal priority
 - **SCHED_RR** - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority
- Defines two functions for getting and setting scheduling policy:
 - `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
 - `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`

POSIX Real-Time Scheduling API

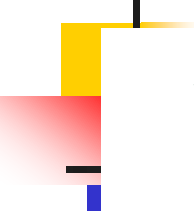
(Fig. 5.25)

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
    int i, policy;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    /* get the default attributes */
    pthread_attr_init(&attr);

    /* get the current scheduling policy */
    if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "Unable to get policy.\n");
    else {
        if (policy == SCHED_OTHER) printf("SCHED OTHER\n");
        else if (policy == SCHED_RR) printf("SCHED RR\n");
        else if (policy == SCHED_FIFO) printf("SCHED FIFO\n");
    }
}
```

POSIX Real-Time Scheduling API (Cont.)



```
/* set the scheduling policy - FIFO, RR, or OTHER */  
if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)  
    fprintf(stderr, "Unable to set policy.\n");
```

```
/* create the threads */  
for (i = 0; i < NUM_THREADS; i++)  
    pthread_create(&tid[i], &attr, runner, NULL);
```

```
/* now join on each thread */  
for (i = 0; i < NUM_THREADS; i++)  
    pthread_join(tid[i], NULL);
```

```
}
```

```
/* Each thread will begin control in this function */
```

```
void *runner(void *param)
```

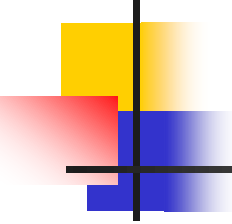
```
{
```

```
    /* do some work ... */
```

```
    pthread_exit(0);
```

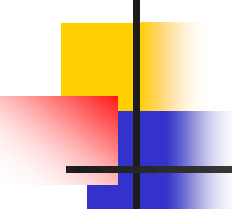
```
}
```





5.7 Operating-System Examples

- **5.7.1 Example: Linux Scheduling**
- **5.7.2 Example: Windows Scheduling**
- **5.7.3 Example: Solaris Scheduling**



5.7.1 Example: Linux Scheduling

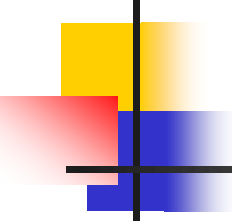
- Prior to kernel version 2.5, ran variation of standard UNIX scheduling algorithm
- Version 2.5 moved to constant $O(1)$ scheduling time
 - Preemptive, priority based
 - Two priority ranges: time-sharing and real-time
 - **Real-time** range from 0 to 99 and **nice** value from 100 to 140
 - Map into global priority with numerically lower values indicating higher priority
 - Higher priority gets larger quantum
 - Task run-able as long as time left in time slice (**active**)
 - If no time left (**expired**), not run-able until all other tasks use their slices
 - All run-able tasks tracked in per-CPU **runqueue** data structure
 - Two priority arrays (active, expired)
 - Tasks indexed by priority
 - When no more active, arrays are exchanged
 - Worked well, but poor response times for interactive processes

List of Tasks Indexed According to Priorities



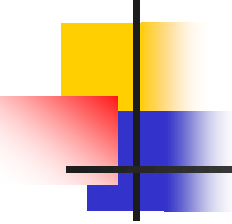
Linux Scheduling in Version 2.6.23 +

- ***Completely Fair Scheduler (CFS)***
- **Scheduling classes**
 - Each has specific priority
 - Scheduler picks highest priority task in highest scheduling class
 - Rather than quantum based on fixed time allotments, based on proportion of CPU time
 - 2 scheduling classes included, others can be added
 1. default
 2. real-time
- **Quantum calculated based on *nice value* from -20 to +19**
 - Lower value is higher priority
 - Calculates **target latency** – interval of time during which task should run at least once
 - Target latency can increase if say number of active tasks increases



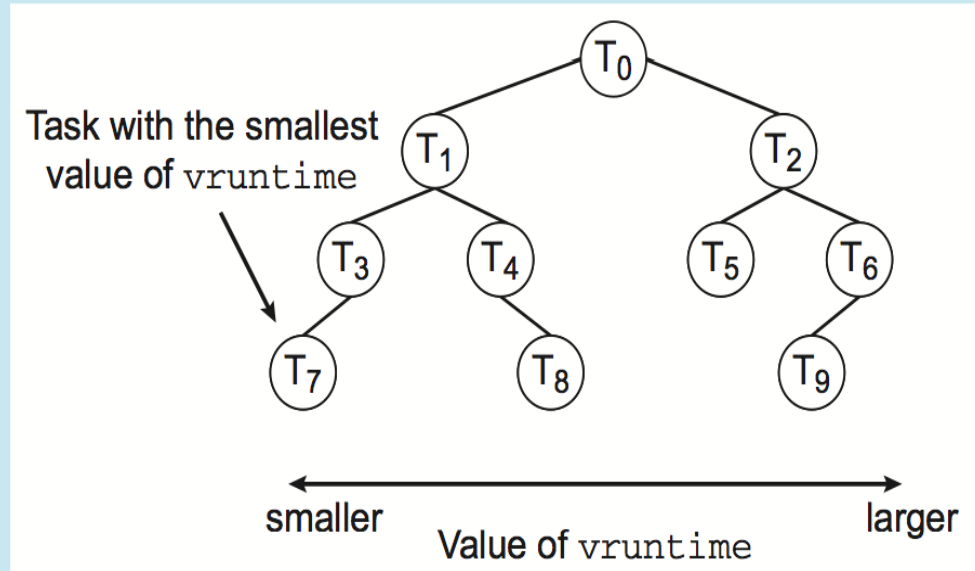
Linux Scheduling in Version 2.6.23 +

- **CFS scheduler maintains per task `virtual run time` in variable `vruntime`**
 - Associated with decay factor based on priority of task – lower priority is higher decay rate
 - Normal default priority yields virtual run time = actual run time
- **To decide next task to run, scheduler picks task with lowest virtual run time**



CFS Performance

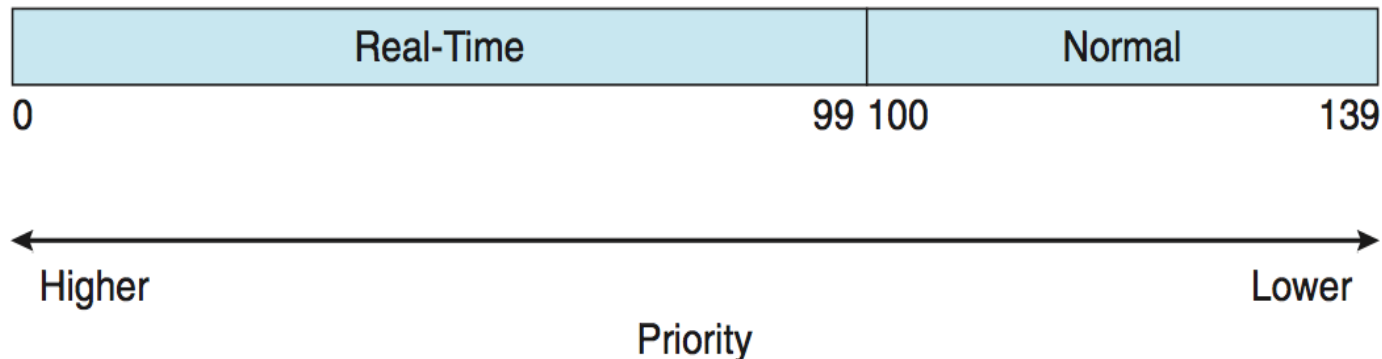
The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

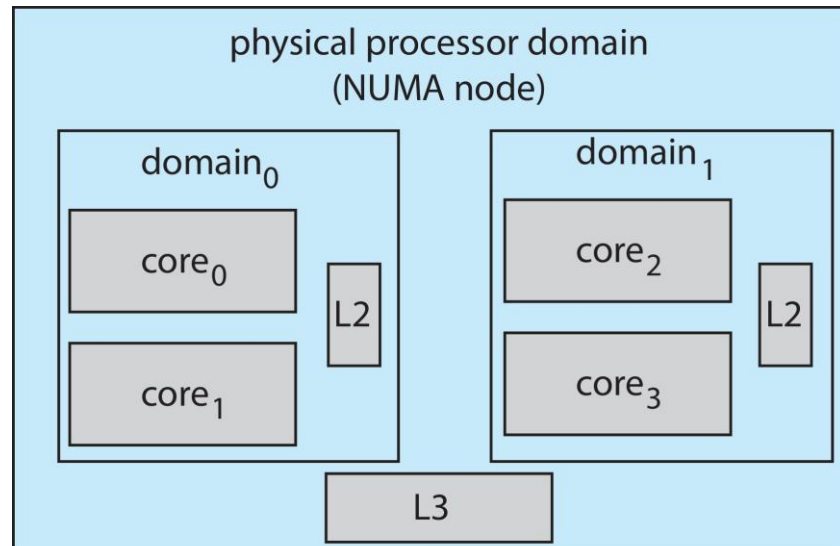
Linux Scheduling (Cont.)

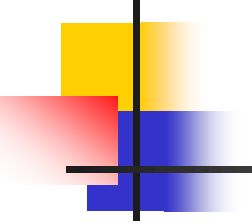
- **Real-time scheduling according to POSIX.1b**
 - Real-time tasks have static priorities
- **Real-time plus normal map into global priority scheme**
- **Nice value of -20 maps to global priority 100**
- **Nice value of +19 maps to priority 139**



Linux Scheduling (Cont.)

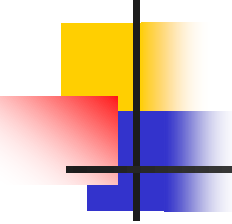
- Linux supports load balancing, but is also NUMA-aware.
- Scheduling domain is a set of CPU cores that can be balanced against one another.
- Domains are organized by what they share (i.e., cache memory.) Goal is to keep threads from migrating between domains.





5.7.2 Example: Windows Scheduling

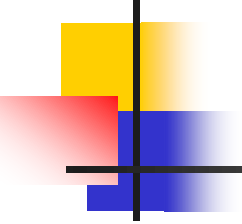
- Windows uses priority-based preemptive scheduling
- Highest-priority thread runs next
- **Dispatcher** is scheduler
- Thread runs until (1) blocks, (2) uses out time slice, (3) preempted by higher-priority thread, (4) terminates
- Real-time threads can preempt non-real-time
- 32-level priority scheme
- **Variable class** is 1-15, **real-time class** is 16-31
- Priority 0 is memory-management thread
- Queue for each priority
- If no runnable thread, runs **idle thread**



Windows thread priorities

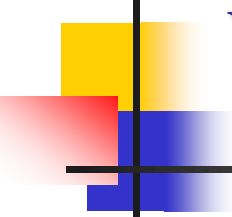
Priority classes

Relative priority		real- time	high	above normal	normal	below normal	idle priority
	time-critical	31	15	15	15	15	15
	highest	26	15	12	10	8	6
	above normal	25	14	11	9	7	5
	normal	24	13	10	8	6	4
	below normal	23	12	9	7	5	3
	lowest	22	11	8	6	4	2
	idle	16	1	1	1	1	1



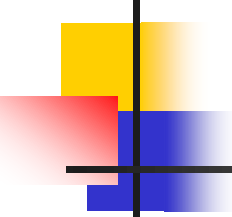
Windows Priority Classes

- **Win32 API identifies several priority classes to which a process can belong**
 - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
 - All are variable except REALTIME
- **A thread within a given priority class has a relative priority**
 - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- **Priority class and relative priority combine to give numeric priority**



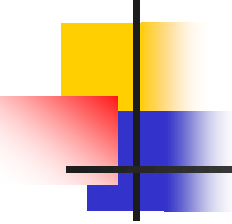
Windows Priority Classes (Cont.)

- Base priority is **NORMAL** within the class
- If quantum expires, priority lowered, but never below base
- If wait occurs, priority boosted depending on what was waited for
- Foreground window given 3x priority boost
- Windows 7 added **user-mode scheduling (UMS)**
 - Applications create and manage threads independent of kernel
 - For large number of threads, much more efficient
 - UMS schedulers come from programming language libraries like C++ **Concurrent Runtime** (ConcRT) framework



5.7.3 Example: Solaris Scheduling

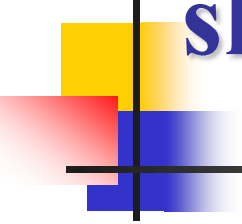
- **Priority-based scheduling**
- **Six classes available**
 - Time sharing (default) (TS)
 - Interactive (IA)
 - Real time (RT)
 - System (SYS)
 - Fair Share (FSS)
 - Fixed priority (FP)
- **Given thread can be in one class at a time**
- **Each class has its own scheduling algorithm**
- **Time sharing is multi-level feedback queue**
 - Loadable table configurable by sysadmin



Solaris Scheduling (Cont.)

- **Scheduler converts class-specific priorities into a per-thread global priority**
 - Thread with highest priority runs next
 - Runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread
 - Multiple threads at same priority selected via RR

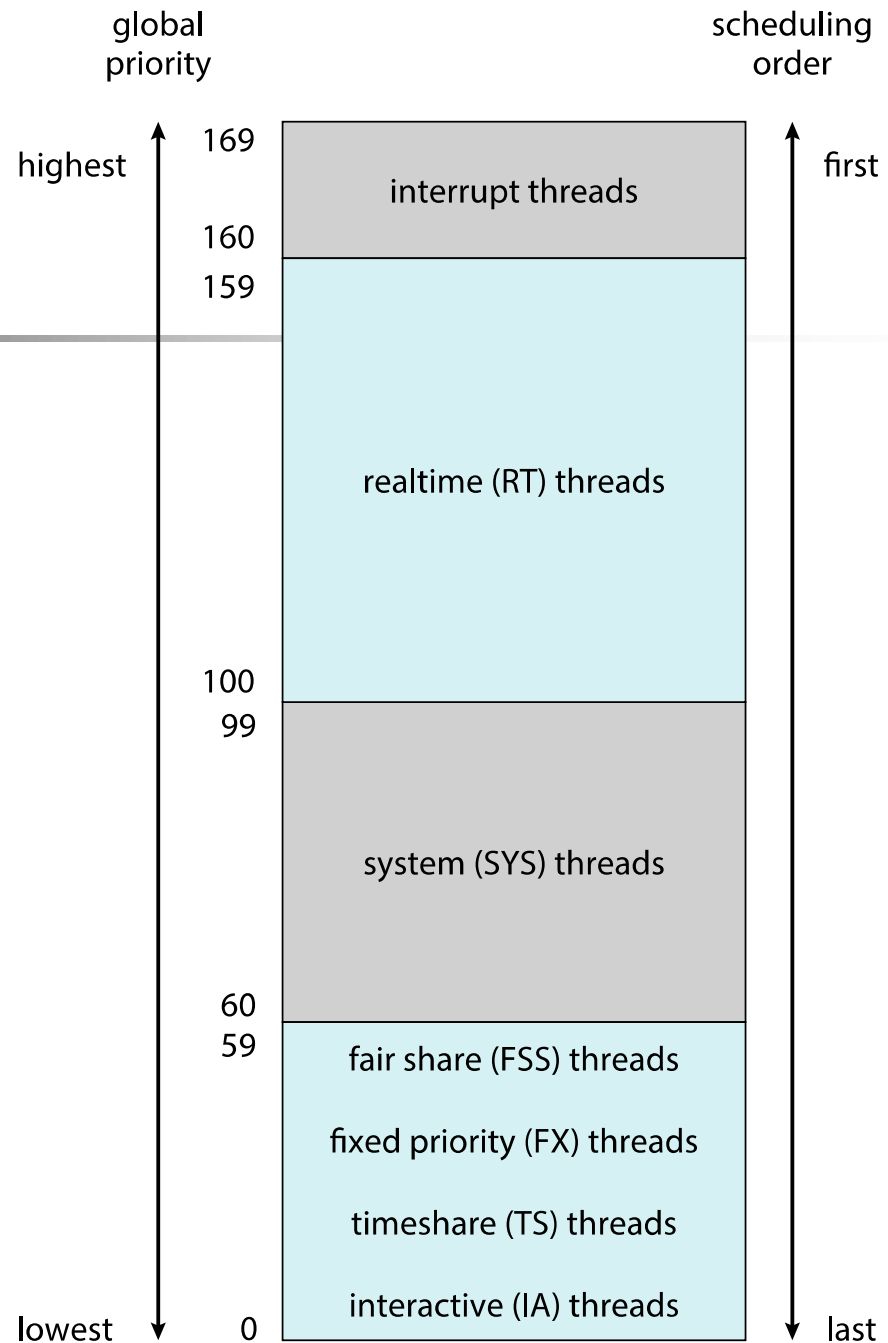
Solaris dispatching table for time-sharing and interactive threads

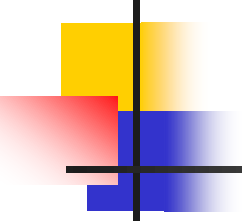


priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

↓
higher priority

Solaris Scheduling





5.8 Algorithm Evaluation

- **Criteria** to select a CPU scheduling algorithm may include several measures, such as :
 - Maximize CPU utilization under the constraint that the maximum response time is 300 ms
 - Maximize throughput such that turnaround time is (on average) linearly proportional to total execution time
- **Evaluation methods:**
 - 5.8.1 Deterministic modeling 
 - 5.8.2 Queuing models 
 - 5.8.3 Simulations 
 - 5.8.4 Implementation 



既定性的，決定性的

5.8.1 Deterministic Modeling

- Takes a **particular predetermined workload** and defines the performance of each algorithm for that workload.
- simple and fast
- gives exact numbers, allows algorithms to be compared
 - requires exact numbers of input, and its answers apply to only those cases
- **too specific**, and requires too much exact knowledge, to be useful
- The main uses are in describing scheduling algorithms and providing examples.

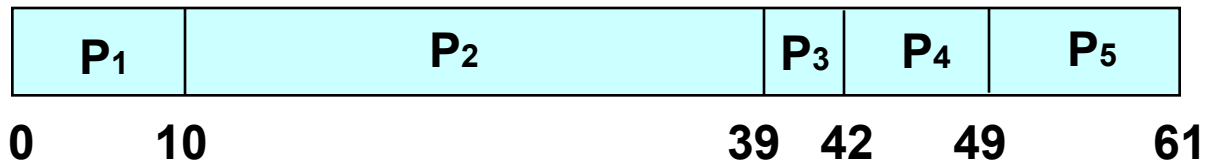
Deterministic Modeling

Example

Process	Burst Time
P1	10
P2	29
P3	3
P4	7
P5	12

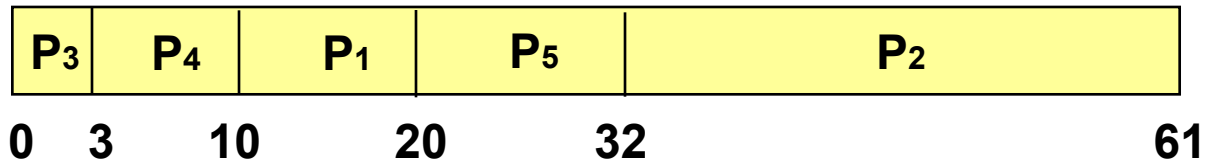
average
waiting time

FCFS



28 ms

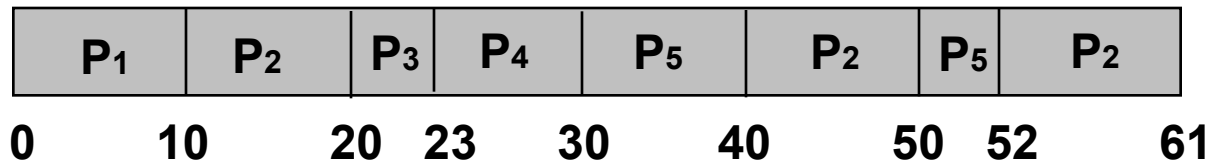
SJF



13 ms

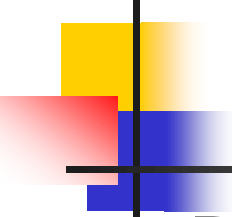
RR

(q=10)



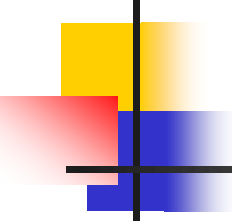
23 ms





5.8.2 Queuing Models

- **Processes that are run on many systems vary from day to day**
 - there is no static set of processes (and times) to use for deterministic modeling
- **What can be determine**
 - the *distribution of CPU and I/O bursts*
 - a *mathematical formula* describing the probability of a particular CPU burst
 - Commonly, this distribution is exponential and is described by its mean.
 - the *distribution of processes arrival time*
- **These distribution may be measured and then approximated or simply estimated.**



Modeling Computer System

- **The computer system is described as a network of servers.**
 - Each server has a queue of waiting processes.
 - The CPU is a server with its ready queue, as is the I/O system with its device queues.
- **Queuing-network analysis**
 - known
 - arrival rates and service rates
 - to compute
 - utilization, average queue length, average waiting time, and so on.

Queuing Analysis

Little's formula:

$$n = \lambda \times W$$

when process arrival rate
= process departure rate

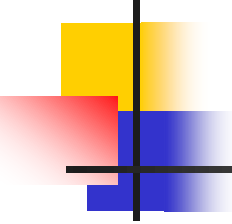
- n : average queue length
- W : average waiting time in the queue
- λ : average arrival rate for new processes
- We can use this formula to compute one of the three variables, if we know the other two.

■ Useful in comparing scheduling algorithms, but has limitations.

- The arrival and service distributions are often defined in unrealistic, but mathematically tractable, ways.

易處理的





5.8.3 Simulations

- Simulations get a **more accurate evaluation** of scheduling algorithms.
- Involve software programming a model of computer system.
 - A simulator
 - Has a variable representing a clock; as the clock is increased, the simulator modifies the system state to reflect the activities of devices, the processes, and the scheduler.
 - Input data
 - Output of statistics

Simulation (Cont.)

■ Input

- pid arrive-time burst-time priority

■ Simulator

■ Output

- Gantt chart
- Average waiting time
- Average turnaround time
- How many context switches
- Total time required to finish all processes

Simulator:

```
clock=0
```

```
While not-end-of-input-records
```

```
{
```

```
    if clock==r.arrive-time
```

```
        RunPolicy // preemptive
```

```
    if clock==r.finish-time
```

```
        RunPolicy // non-preemptive
```

```
                // preemptive
```

```
    clock++;
```

```
    ...
```

```
}
```

Each scheduling object:

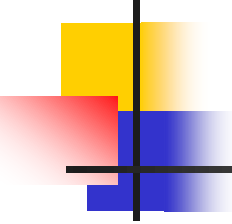
```
Class schedule
```

```
{
```

```
    checkinthread();
```

```
    checkoutthread();
```

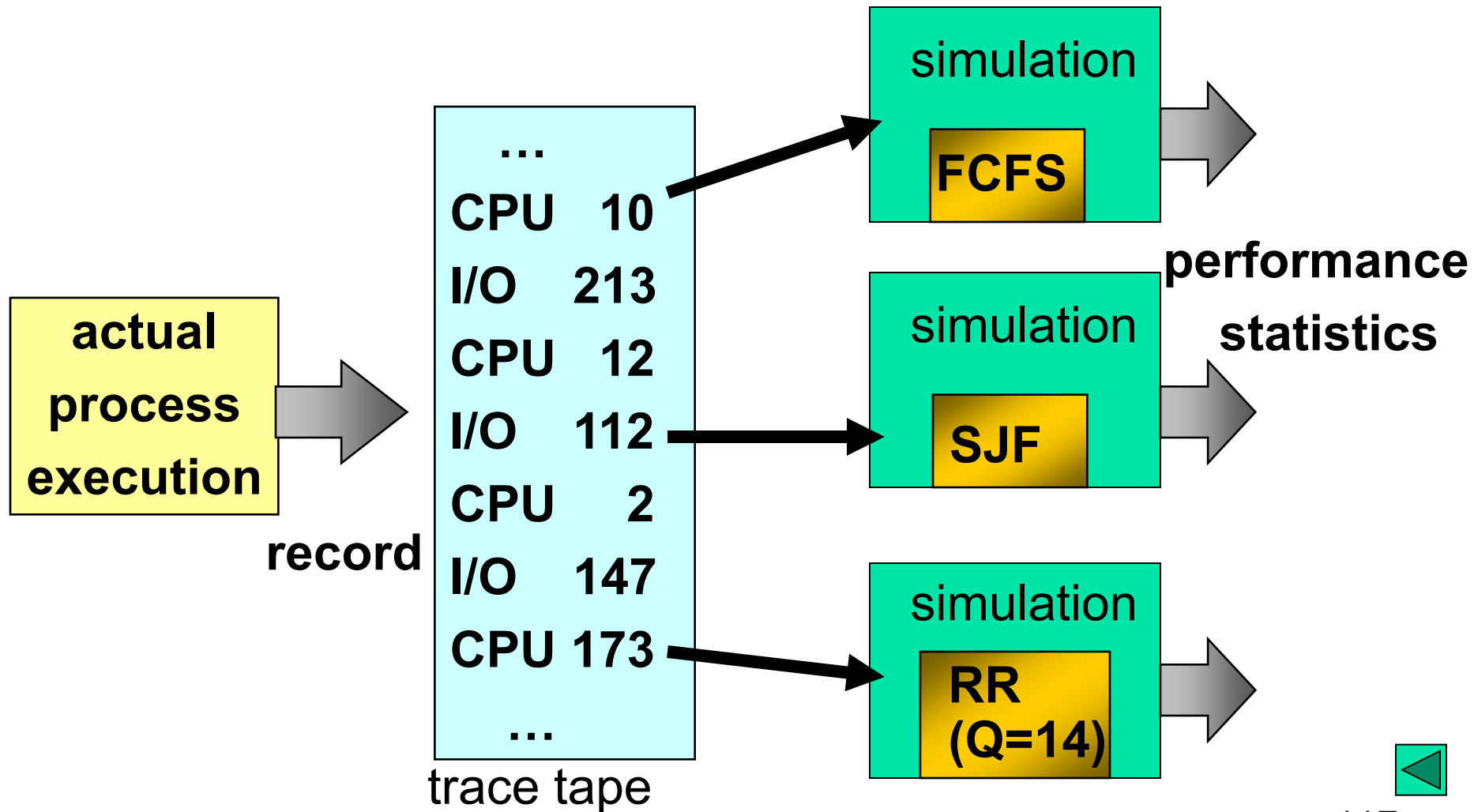
```
}
```

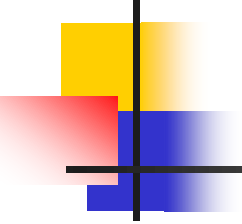


Simulation (Cont.)

- **Data to drive the simulator can be generated in several ways:**
 - **distribution-driven simulation**
 - randomly generates data
 - the distribution may be defined mathematically or empirically 憑經驗地
 - **simulation with trace tapes**
 - A trace tape can be created by monitoring the real system, recording the sequence of actual events.

Evaluation of CPU Schedulers by Simulation





5.8.4 Implementation

- **The only completely accurate way to evaluate a scheduling algorithm is to code it up in the OS, and to see how it works.**
- **This approach puts the actual algorithm in the real system for evaluation under real operating conditions.**
- **Major difficulty: high cost.**



NCNU IM OS Simulator

<http://ossim.im.ncnu.edu.tw:8080/>

[How to Use?](#)

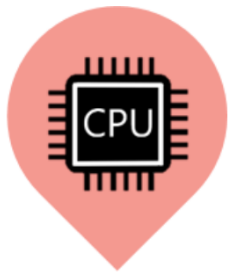
 [Q&A](#)

[About](#)



[Joanna](#)

[Log out](#)



CPU
Scheduling



Real-Time
Scheduling



Deadlock



Page
Replacement



Disk
Scheduling

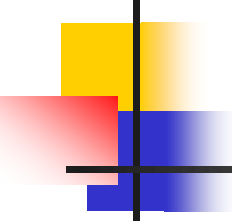


Operating System Online Simulator



Advisor: Mei-Ling Chiang, Professor

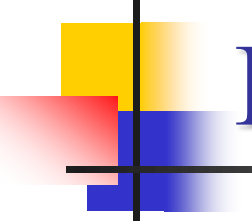
Developers: Ting-Yu Su, Sih-Bei Chen, Yun-Han Hsu, Hsiao-Tung Yang, Run-An Yeh, Jou-En Chien
Department of Information Management, National Chi Nan University, Nantou, Puli, Taiwan, R.O.C.



Homework

■ Textbook

- 10th edition: ex 5.3, (5.4), 5.5, 5.17, 5.18, (5.35)
 - (Ex. 5.4) Show the Gantt chart for scheduling, the waiting time for each process, the average waiting time
 - (Ex. 5.35) For real-time scheduling, show the Gantt chart for scheduling
 - 對照組: 繳交手寫作業
 - 實驗組: 使用OS Simulator，用自己的帳號，繳交執行結果螢幕截圖 (<http://ossim.im.ncnu.edu.tw:8080/>)



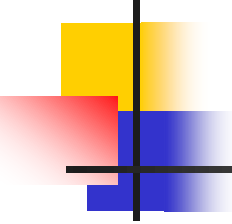
Ex. 5.4

5.4 Consider the following set of processes, with the length of the CPU burst time given in milliseconds:

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

The processes are assumed to have arrived in the order P_1, P_2, P_3, P_4, P_5 , all at time 0.

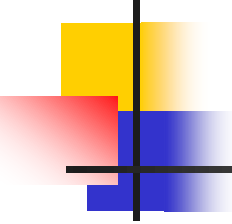
- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting time (over all processes)?



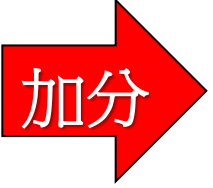
Ex. 5.35

5.35 Consider two processes, P_1 and P_2 , where $p_1 = 50$, $t_1 = 25$, $p_2 = 75$, and $t_2 = 30$.

- Can these two processes be scheduled using rate-monotonic scheduling? Illustrate your answer using a Gantt chart such as the ones in Figure 5.21–Figure 5.24.
- Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling.



Homework (加分題)

- 
- 加分題: **Write a simulation program to implement preemptive CPU scheduling algorithms.**
 - Refer to slides 5.8.3 Simulations

注意! 1. 請印出執行結果畫面 & 交原始程式
2. 請找**TA Demo** (沒有**Demo**不給分)